UTRECHT UNIVERSITY

Department of Information and Computing Sciences

Faculty of Sciences

---

alliGATOR : Graph Anomaly Detection through diffusion based Topology Reconstruction

---

A master thesis submitted in fulfilment of the requirements for the degree of Master of Science in Artificial Intelligence

Yulia Ivanova Terzieva

Daily supervisor: PhD Candidate R. Rico

1st supervisor: Dr. I.R. Karnstedt-Hulpus

2nd supervisor: Prof. C.W. Oosterlee

June 16th, 2025

Abstract

Financial crime detection concerning monetary transactions remains a critical challenge in the fight against illicit activities such as fraud and money laundering. In this thesis, we present the alliGATOR model, an unsupervised graph anomaly detection model designed to identify both node and edge anomalies. Our approach is grounded in the notion that node connections are dependent on their attributes. We hypothesised that anomalies can be detected by comparing a graph's original topology to one reconstructed from node attributes alone.

The alliGATOR model requires a novel node-guided topology reconstructor model – a discrete generative diffusion model based on a graph neural network link predictor. The model iteratively reconstructs the graph structure by adding edges conditioned on the node attributes and degrees, generating a topology that obeys the local properties of the nodes. Our node-guided topology reconstructor operates under an inductive setting and can generalise across graphs.

We evaluate the alliGATOR model on a custom synthetic dataset designed to simulate real-world behaviour, and compare its performance to the Isolation Forest baseline. Experimental results show that alliGATOR is robust and successfully identifies both node and edge anomalies.

This work establishes a foundation for applying generative diffusion models to graph anomaly detection, and it can support researchers, financial institutions, banks, and governments in developing personalised autonomous anomaly detection systems.

Acknowledgements

To Dr. Ioana Karnstedt-Hulpus – This project would not have been possible without your stalwart guidance and patience. I am deeply honoured and grateful to have had the opportunity to work under your supervision over the past few months. I have learned so much from you – not just in theory, but also in approaching research with professionalism, curiosity, and optimism. It was truly a pleasure collaborating with you; I looked forward to every weekly meeting and each brainstorming session, where we explored all the "ifs" and "ands", and discussed the countless ways of improving the alliGATOR.

To Prof. Kees Oosterlee – Your Neural Networks for Finance course ignited my interest in diffusion models and their applications in the financial domain. Without you, I would not have taken this exciting research path. Thank you for your critical questions and valuable feedback throughout the process.

To my partner – You are the rock that kept me afloat during the thesis. Thank you for celebrating every small victory with me, from successful experiments to tiny breakthroughs, and for offering unwavering comfort and confidence during the moments I doubted myself. I could not have done this without you.

To my parents – Благодаря Ви за безкрайната подкрепа и любов през последните шест години. Вие ми дадохте възможността и увереността да работя по най-интересните и предизвикателни изследователски въпроси, за което съм Ви вечно благодарна!

CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ALGORITHMS

CHAPTER

ONE

INTRODUCTION

People spend more than six hours a day online. The information, which is shared willingly, such as names, emails, and dates of birth, as well as the information that is extracted from the online activity, such as preferences, political views, and opinions, is used to create individual profiles. This information shared online makes people vulnerable to criminals, who may impersonate others and target personal finances (Sumsub, 2024). Using the individual profiles, people susceptible to pressure and blackmail may be forced into money laundering schemes. Criminal activities, such as identity theft, fraudulent transactions, tax evasion, and terrorist funding, cost us billions of euros a year (EBA and ECB, 2024; Cifas and Royal United Services Institute, 2024).

Criminal behaviour is detected by banks and financial institutions. These organisations have monitoring systems in place that analyse monetary transactions and flag suspicious individuals, businesses, groups, and transactions. Entities are suspicious when they are anomalous, and they are anomalous when they have activity out of the ordinary.

From a technical perspective, monetary interactions can be represented as graph-structured data; where nodes typically correspond to bank account holders and edges represent transactions. These graphs may be attributed, with node attributes capturing the account holder's typical pattern of activity, and personal information, such as age, geolocation and income. Edge attributes may include transactional amount, frequency of edge occurrence, and time of day. Therefore, an anomalous edge would be between accounts that do not typically interact, or represents a transaction that deviates significantly in amount, frequency, or time of day compared to historical patterns. A node would be considered anomalous when it presents with atypical activity to the expected, based on its attributes.

There are different methods for detecting anomalies in graph data. Supervised and weakly supervised approaches, often rely on graph embeddings to determine whether entities of interest (nodes, edges, or subgraphs) are anomalous (Jiang et al., 2023). These embeddings are usually generated by pre-trained neural networks designed to work with graph-structured data. However, for the training of the classifiers and the underlying embedding models, labelled datasets are required. This proves problematic as data labelling requires manual labour from domain experts. In the financial context, although banks and financial governmental institutions may have people tasked with labelling fraudulent instances, privacy protection laws prohibit them from sharing such information externally.

Unsupervised methods offer an alternative, and often a complementary, solution to supervised approaches, as they do not require labelled data, or, in many cases, domain-specific knowledge for their training (Goldstein and Uchida, 2016). There are different methods to detect anomalies without supervision, however, if the graph attributes do not relay sufficient information, graph embeddings are still required. There are methods of computing the embeddings without labelled data, for example using random walks. Regardless of the methodology used for calculating the embeddings, there are various unsupervised anomaly detection algorithms that can be used, such as Isolation Forest (Liu et al., 2008). The algorithm works by recursively splitting the input data into smaller subsets, and identifying anomalies based on how few splits are required to isolate a given instance from the rest of the data. The splitting is based on the attribute values of the instances. Recent research explores the use of generative models for anomaly detection (Xia et al., 2022; Wolleb et al., 2022). The underlying assumption is that generative models learn to generate normal data, therefore, any deviations from what they have generated can be considered anomalous. It is presumed that learning the underlying distribution of anomalous samples is harder, and due to the minimal available instances to learn from, the generative approaches model only the normal data.

Well-known classes of generative models include variational auto-encoders (Kingma and Welling, 2014), generative adversarial networks (Goodfellow et al., 2020), and diffusion models (Ho et al., 2020). Among these, diffusion models represent the most recent and advanced approach, addressing key limitations of earlier methods – such as the instability and mode collapse often observed in generative adversarial networks and the limited expressiveness of variational auto-encoders. Diffusion models learn to reverse the incremental addition of noise to data, which allows them to generate data from pure noise, such that the generated data follows the same distribution as the original one (Ho et al., 2020).

Although diffusion models have been introduced relatively recently, there is already a growing body of research on their application to graph generation. The proposed approaches can be partitioned in two ways. The first is based on the graph component being diffused and generated – namely, the node attributes, the edge structure (i.e. the adjacency matrix), or a combination of both. The second way of partitioning the work is based on the nature of the diffusion process – whether it relies on continuous noise or discrete noise for the training and sampling of the data. Prominent works include those of Niu et al. (2020) and Huang et al. (2022b), both of which focus on the generation of the graph's adjacency matrix, and the work of Jo et al. (2022), which targets both the node features and the graph edges. Those three works use continuous noise to perturb the graph's components, which may be considered unsuitable given that the graph's topology is often inherently discrete. To address this, Vignac et al. (2023) and Haefeli et al. (2022) are the first to introduce discrete diffusion to graphs. The discrete diffusion models use discrete distributions to perturb the graph instead of continuous noise. Initial models for graph generation using diffusion were restricted to generating a few hundred nodes, due to high computational complexity. However, new research has proposed a solution to overcome this. The model by Chen et al. (2023), which is a discrete diffusion model applied to the adjacency matrix, generates graphs by first sampling nodes, then partitioning the nodes across different time steps, and predicting edges only between the "active" nodes on each time step. This idea of predicting edges only between a subset of selected nodes, instead of all nodes, decreases the computational load on the predictive model, therefore allowing for the generation of graphs with up to a few thousand nodes.

Using graph generative models for anomaly detection is not a straightforward task. The models introduced above, alongside classical network models designed to generate realistic synthetic graphs – such as the Erdős–Rényi model (Erdős and Rényi, 1961), the configuration model (Newman, 2018), and the Barabási–Albert model (Barabási and Albert, 1999) – aim at capturing the global, macro-level properties of the graphs. However, to detect anomalies on the node, edge, or subgraph level, the focus needs to be on modelling the local, micro-level properties of the instances of interest.

To model the local properties, the generation process needs to be contextualised, which is done by conditioning it on one of the graph components – either the node attributes or the topology of the graph. Therefore, modelling the nodes' pattern of connectivity can be done using a diffusion model that generates graph topology guided by the node attributes. Such a model can be used to detect node-level anomalies, under the assumption that node connections are dependent on their attributes. Alternatively, node attributes can be modelled by a generative model conditioned on

the graph topology. This approach would operate under the assumption that node attributes are dependent on their connectivity.

There is one work that proposes a model in line with this idea. Li et al. (2023) suggest an anti-money laundering method that reconstructs noised node attributes using a continuous diffusion model, guided by the topology. They calculate a money laundering score based on the difference between the original and the reconstructed node features.

In this work, we propose the alliGATOR model, a graph anomaly detection model based on a node-guided topology reconstructor. Our model keeps the node attributes the same and uses them as a guiding mechanism during graph topology reconstruction. The original and the reconstructed versions of the graph are subsequently compared, based on which an anomaly score is calculated. Our model operates under the assumption that the node degrees are intrinsic to the nodes, and that node connections are dependent on their attributes and degrees. Our proposed model is unsupervised and can be used for both node and edge anomaly detection; with just a slight variation of the input preprocessing and the anomaly calculation. During inference, the input of the model is a graph with node attributes, and the output of the model is an anomaly score for each instance of interest; either node or edge.

The alliGATOR model has three phases. The first one is the preprocessing of the input data – for each instance of interest, a subgraph of the most important nodes for that instance is induced. During the second phase, the edges of the subgraph are removed and generated by a pre-trained diffusion model for topology reconstruction. The last phase is the anomaly score calculation, for which the original and the reconstructed graph topologies are compared.

There are no diffusion models designed to model local graph properties by reconstructing the graph topology for a given set of nodes. Therefore, we adapted a general graph generation model, that focuses on adjacency matrix generation through diffusion. The original model, henceforth called EDGE, is presented by Chen et al. (2023), and we chose it because of its scalability and discrete approach to diffusing the graph topology. We adapt the model by adding classifier-free node-guidance to the generation process. Using the proposed topology reconstructor in the alliGATOR model we aim to answer the following research question :

Can node-guided topology reconstruction be used for graph anomaly detection?

Answering this question requires two experiments addressing the following sub-questions:

SQ1: Can node anomalies be detected by comparing the original induced subgraph with a reconstructed one, that is generated by a diffusion model guided by the original nodes' attributes and degrees?

SQ2: Can edge anomalies be detected by comparing the original induced subgraph with a reconstructed one, that is generated by a diffusion model guided by the original nodes' attributes and degrees?

To answer these questions, we generate two synthetic datasets, one for each type of anomaly, and inject anomalies such that they account for 4% of the instances. We then train the alliGATOR model in an inductive setting, and evaluate its performance by presenting the precision-recall curve and reporting the average precision. To contextualise the results, we benchmarked our model's performance against that of the Isolation Forest algorithm.

This research work has two contributions. The first one is the introduction of a new unsupervised graph anomaly detection model, designed to detect anomalies on node and edge level by comparing an original graph topology with a reconstructed topology. We call this model the alliGATOR. The second contribution is a node-guided topology reconstruction model, which is a diffusion model designed to learn a graph's local properties. The proposed topology reconstructor is used in the alliGATOR model.

The rest of the report is structured as follows: Chapter 2 presents the related work, which includes the mathematical definition of a graph, examples of classical graph generation models, approaches to subgraph extraction, graph neural networks, the theory behind diffusion models, and graph diffusion models. It further delves into the details behind the original model by Chen et al. (2023). Then it presents anomaly detection methods and the use of diffusion models for graph anomaly detection. The Chapter concludes with a discussion on the relation between graph anomaly detectors and link predictors. Chapter 3 presents the alliGATOR model. It includes the formulation of the model, the proposed node-guided diffusion model, the preprocessing of the input for the two variations of the model, the anomaly score functions, and the model implementation and analysis. Chapter 4 includes the data generation, the evaluation metrics, the models parameter settings, and presents and discusses the results of the experiments. The conclusion of the work is presented in Chapter 5.

CHAPTER

TWO

RELATED WORK

This work presents a novel graph anomaly detection method, that works by comparing a graph's original topology with its reconstructed topology. The topology is reconstructed by a diffusion model, which uses a link predictor to generate the edges on each step. Therefore, this section provides an overview of relevant terminology and previous work conducted by other researchers. The three foundational concepts are graphs, diffusion models, and anomaly detection methods. Our proposed model is based on a specific diffusion model for graph topology generation, and for the understanding of the methodology presented in the next chapter, the workings of the original model have to be discussed in detail. Furthermore, graph topology generation is closely related to link prediction; therefore, the related work would not be exhaustive without a discussion of the research in this area. Thus, this chapter is divided into: Graphs (Section 2.1), Graph Representation Learning (Section 2.2), Diffusion Models (Section 2.3), the EDGE model (Section 2.4), Anomaly Detection (Section 2.5), and Link Predictors (Section 2.6).

2.1 Graphs

A graph is mathematically defined as a tuple $G = (V, E)$ such that $V = \{v_1, ..., v_n\}$ is a set of nodes and $E = \{\{e_{v_i,v_j}\} | v_i, v_j \in V\}$ is a set of edges. Let's denote the number of nodes with $n = |V|$ and edges with $m = |E|$. A graph can be represented by an adjacency matrix $A \in \{0,1\}^{n \times n}$, where $A_{ij} = 1$ if there is an edge from node v_i to node v_j, and 0 otherwise. Depending on the graph type, other variations of the adjacency matrix exist – for example, multiedge or weighted graphs

have real-valued adjacency matrices. Graphs often contain node and edge attributes, which capture additional information. As an example, node attributes can be represented with a matrix $X \in \mathbb{R}^{n \times d}$ with d the node attribute dimension.

2.1.1 Graph Generation Models

There exist models of graph generation that mimic the patterns of connections in real-world graphs. These models are used with the intent of studying the effects different parameter settings have on the resulting networks. Those experiments are essential for a better understanding of real-world networks. Additionally, these models play an important role in providing a foundation for the exploration of different processes that occur in graphs, such as disease spread (Newman, 2018).

The Erdős–Rényi model (Erdős and Rényi, 1961) is one of the earliest such models. It is a random graph model, meaning that some properties of the graph are known and fixed, but the graph overall is random. The Erdős–Rényi model takes two parameters, the number of nodes n, and a probability p, and produces a graph in which each possible edge exists independently, with probability p. The resulting graph has a binomial degree distribution.

The configuration model (Newman, 2018) is a more sophisticated random graph model that accounts for node degree variation. It allows for a predefined degree sequence for the nodes. The input is, therefore, the number of nodes n and a degree sequence vector, and the output is a graph with randomly connected nodes with preserved degree sequence.

A more realistic graph generation model is the Barabási–Albert model (Barabási and Albert, 1999), which introduces preferential attachment. Their model is of growing graphs, it takes as input two variables – n the number of total nodes in the graph, and c the number of connections each node creates upon addition. Nodes are added bit by bit to the graph and are connected to c previously existing nodes, with the probability of connecting to each node being proportional to the existing nodes' degrees. The resulting graph has a power-law degree distribution, which makes the model more realistic, as this degree distribution is more commonly seen in real-world networks compared to the binomial distribution.

2.1.2 Subgraph Extraction

A subgraph $G' = (V', E')$ of a graph $G = (V, E)$ is defined by subsets $V' \subseteq V$ and $E \subseteq E'$, where each edge in E' connects only nodes from V'. Subgraphs are used for various graph analyses, for example when the research interest is on an instance level, with a goal of exploring and analysing

the instance's role and connectivity type within its immediate network; also referred to as "local properties". A subgraph around a central instance of interest can be induced of all the neighbouring nodes, referred to as a one-hop ego-network, or induced of all K-hop neighbours of the instance. Alternatively, one might induce a subgraph only of the important neighbouring nodes, therefore requiring ranking the nodes based on "importance".

Personalised PageRank (PPR), an extension of PageRank (Page et al., 1999), is a widely known algorithm used for ranking nodes based on their importance to a central node. PageRank calculates a score value for all nodes in a directed graph using Eq 2.1, where $\mathbf{x}$ is a vector of node's ranking, $\mathbf{A}$ is the adjacency matrix, $\mathbf{D}$ is a diagonal matrix with elements $Dii = max(k_i^{out}, 1)$, k_i^{out} is the outgoing node degree of node i, α is a constant, and s is a "teleportation" vector. The teleportation vector has a length equal to the number of nodes in the graph and is uniformly set to $1/|V|$ for each node. It is used to solve problems such as dead ends (nodes without outgoing links) and spider traps (subgraphs without external outgoing links). The node rankings are computed iteratively until convergence. To make the PageRank personalised to node c, means to calculate the ranks of all nodes with respect to c. This is achieved by setting the teleportation vector s as a one-hot at the personalised node c.

$$\mathbf{x} = (1 - \alpha)\, \mathbf{s} + \alpha \mathbf{A}\mathbf{D}^{-1}\mathbf{x} \tag{2.1}$$

2.2 Graph Representation Learning

Graph-structured data is used in domains where the relations between constituent data instances convey crucial information for the understanding of the data as a whole – for example, in social networks, biological molecules, and knowledge graphs. To apply machine learning methods to such data, especially when the graph is unattributed, one must derive useful representations. This can be done through hand-crafted graph features, such as node degree, clustering coefficient, and various centrality measures (Newman, 2018), or by learning graph embeddings.

Graph embedding methods aim to map nodes, edges, or entire graphs into a continuous vector space, preserving structural or relational properties. Classical algorithms for node embedding generation are node2vec (Grover and Leskovec, 2016a) and struc2vec (Ribeiro et al., 2017). The node2vec model embeds nodes such that nodes that have similar neighbourhoods also have similar embeddings, whereas the struc2vec model creates embeddings that focus on structural similarity of

nodes. For whole-graph embeddings, models like graph2vec (Narayanan et al., 2017) represent entire graphs by aggregating information from their constituent nodes. While these traditional embedding techniques have been successfully applied to tasks such as node classification, link prediction, and graph classification, they typically do not incorporate node or edge attributes directly and require separate preprocessing pipelines.

This limitation motivated the development of graph neural networks, the first such model introduced by Kipf and Welling (2017). These are models that operate directly on graph-structured data and learn embeddings in a task-specific manner. The work by Hamilton et al. (2017) is pivotal in this field. The reason is that they introduced an inductive framework for node embedding based on graph neural networks, thus addressing the main disadvantage of the previous models – they require all nodes to be present during training. Inductive graph neural networks leverage node features to learn an embedding function (denoted as f_θ). During inference, the model updates the node attributes by aggregating them with the information (i.e. the attributes) of their neighbours. This operation is repeated at every layer of the network. The input of each layer includes two parameters – (i) a set of node representations $\{h_i \in \mathbb{R}^d | i \in V\}$, where d denotes the dimension, and (ii) a set of edges E. The output of the layer is the new set of node representations $\{h'_i \in \mathbb{R}^{d'} | i \in V\}$. The operation presented in Eq. 2.2 is applied to every node given its direct neighbours. Let the direct neighbours be defined as $\mathcal{N}_i = \{j \in V | (j,i) \in E\}$, where $\mathcal{N}_i$ is the set of nodes that are direct neighbours of node v_i, meaning there is an edge between v_i and the nodes. The input of the first layer are the original node attributes.

$$h'_i = f_\theta(h_i, \text{AGGREGATE}(\{h_j | h_j \in \mathcal{N}_i\})) \tag{2.2}$$

What sets different graph neural network models apart is their choice of functions AGGREGATE and f_θ. The common variant of the GraphSAGE model, proposed by Hamilton et al. (2017), performs element-wise mean as AGGREGATE, and function f_θ concatenates the result of the aggregator with the input node representations h_i and uses a linear layer with ReLU activation function.

Inspired by attention mechanisms, Veličković et al. (2017) propose a graph neural network that incorporates attention during the neighbour aggregation stage, instead of weighing all of the neighbours with equal importance. Their attention graph neural network model is further improved by Brody et al. (2022). The improved version has a scoring function $e : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$ presented in Eq.2.3, which is used to compute a score for every edge (j,i). The score is called *attention coefficient* and it indicates how important the features of neighbour j are to node i. In the equation, both $a \in \mathbb{R}^{d'}$ and $W \in \mathbb{R}^{d' \times 2d}$ are learnable and $\|$ denotes vector concatenation.

$$e(h_i, h_j) = a^\top \text{LeakyReLU}(W \cdot [h_i \| h_j]) \tag{2.3}$$

The attention scores are further normalised over all neighbours of node i, $j \in \mathcal{N}_i$, using softmax, resulting in an attention function presented in Eq. 2.4. The normalised attention coefficients are used to compute the new node representations, effectively replacing Eq. 2.2 with Eq. 2.5. Presented in Eq. 2.5 is one layer of a graph attention network with one attention head, where σ is a non-linear function. However, multiheaded attention can be employed, as presented in Eq. 2.6, where K denotes the number of attention heads. The results of each attention-induced embedding are concatenated to make the new node representation.

$$\alpha_{ij} = \text{softmax}_j(e(h_i, h_j)) \tag{2.4}$$

$$h_i' = \sigma\Big(\sum_{j \in \mathcal{N}_i} \alpha_{ij} \cdot W h_j\Big) \tag{2.5}$$

$$h_i' = \Big\Vert_{k=1}^{K} \sigma\Big(\sum_{j \in \mathcal{N}_i} \alpha_{ij}^k W^k h_j\Big) \tag{2.6}$$

Since their introduction, graph neural networks have been used in various application domains due to their ability to jointly model the graph structure and features. Notable graph-related downstream tasks for which graph neural networks have been explored include models that detect money-laundering activity in transactional graph data (Chen et al., 2023), algorithms that generate possible molecule structures (Jo et al., 2024), and recommender systems used by streaming services like Netflix and Disney (Wu et al., 2022).

2.3 Diffusion Models

Diffusion models are a class of generative models, similar to variational autoencoders (Kingma and Welling, 2014), and generative adversarial networks (GANs) (Goodfellow et al., 2020). Such generative models work by defining a probability distribution $p(z)$ over a latent variable z, which is then transformed by a deep neural network into the data space of the original data x (Sohl-Dickstein et al., 2015; Ho et al., 2020). Put simply, a diffusion model learns to reverse the incremental addition

of noise to data. By learning to do this, the model can generate new data, which follows the same distribution as the original data, from noise. Looking at the sequence of states in the Figure 2.1, x is the original data and $z_1, \ldots, z_T$ are the latent variables. In this example, z_T is a sample from a Gaussian distribution $z_T \sim \mathcal{N}(0, \mathbf{I})$ and the image is incrementally distorted over T diffusion steps. Once the model is trained, it can produce images from Gaussian samples.

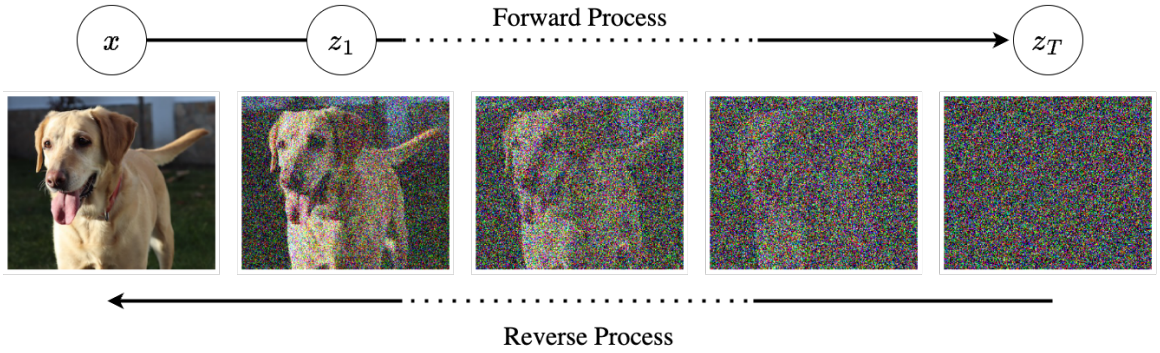


Figure 2.1: Illustration of the forward process of diffusing a picture with Gaussian noise. The reverse process is done by a deep neural network.

Regarding the transitions happening between two consecutive states, we can find that there are two processes. The first one is a **forward transition** that goes from latent state z_{t-1} to z_t with associated transition distribution of $q(z_t|z_{t-1})$, which is known and defined by the modeller. Using the example from Figure 2.1, $q(z_t|z_{t-1})$ would be conditional Gaussian distribution. That means that on each step of the diffusion process, to each pixel of the image a small value would be added, this amount of added "noise" is governed by a *noise scheduler*. Formulated, this can be expressed by $z_t = \sqrt{1-\beta_t} z_{t-1} + \sqrt{\beta_t}\epsilon_t$, where β is the noise scheduler and ϵ is the noise, in this case $\epsilon \sim \mathcal{N}(\epsilon|0, \mathrm{I})$. Therefore, the forward transition for the running example is the conditional Gaussian distribution with mean of $\sqrt{1-\beta_t} z_{t-1}$ and variance of $\beta_t \mathrm{I}$: $q(z_t|z_{t-1}) = \mathcal{N}(z_t|\sqrt{1-\beta_t} z_{t-1}, \beta_t \mathrm{I})$. The forward process forms a Markov chain, and the joint distribution of the latent variable states is expressed in Eq. 2.7.

$$q(z_{1:T}|\mathbf{x}) = q(z_1|x) \prod_{t=2}^{T} q(z_t|z_{t-1}). \tag{2.7}$$

The other process that happens between two consecutive states is a **reverse transition** from state z_t to z_{t-1}. We cannot invert the forward distribution, because it is intractable, as we do not know the distribution of x : $p(x)$. The goal is to model $p(x)$. The solution is to approximate the

reverse distribution using a deep neural network $p_\theta(z_{t-1}|z_t)$. In the running examples with the images and the Gaussian noise, the reverse transition is also modelled by a Gaussian distribution. Therefore, the deep neural network learns to predict the mean of the Gaussian distribution $\mu(z_t, w, t)$ given the current state z_t, the time step t and the parameters w : $p_\theta(z_{t-1}|z_t) = \mathcal{N}(z_{t-1}|\mu(z_t, w, t), \beta_t \mathrm{I})$. The reverse process also forms a Markov chain, and the joint distribution is expressed in Eq. 2.8.

$$\begin{aligned} p_\theta(x, z_{1:T}) &:= p(z_T) p_\theta(x|z_1) \prod_{t=2}^{T} p_\theta(z_{t-1}|z_t) \\ p(z_T) &:= q(z_T) \end{aligned} \tag{2.8}$$

Depending on the application, the distributions used for modelling the transitions are different. In Section 2.4, a diffusion model with a Bernoulli distribution is discussed. Training deep learning models is generally done by maximising the likelihood of the data given the parameters; however, in this case, the likelihood is intractable. Another approach is to maximise the lower bound of the log likelihood, known as evidence lower bound (ELBO) or variational lower bound. This is the approach diffusion models use to optimise the model parameters. An example of this training procedure is described in detail in Section 2.4.

2.3.1 Diffusion Model Guidance

Diffusion models, as presented in the previous Section 2.3, learn how to generate data from unconditional distribution $p(x)$, however, many applications require sampling from conditional distribution $p(x|c)$, where the condition c may be a class label, or a text description of an image that is embedded. If we have data x which consists of pictures of dogs and cats, we want to be able to choose what picture to generate – of a dog, or of a cat. Simply adding the condition c as an extra parameter to the denoising network and training on pairs $\{x_n, c_n\}$ is not enough. The reason is that this approach does not allow us to monitor and influence the importance, i.e. weight, the network gives to the conditioning variable c. There are two methods to *guide* a denoising process using a condition – by employing classifier guidance or a classifier-free guidance.

Dhariwal and Nichol (2021) propose ***classifier guidance***, a method of conditioning the denoising process on variable c. For that, they train a classifier p_ϕ to predict the probability of the condition given noisy samples of the data. For a training pair $\{x_n, c_n\}$, where x_n is an image and c_n is a label, they train the classifier $p_\phi(c_n|z_n^t, t)$ to correctly predict the class of the image, which is noised to a diffusion step t. During image generation, the gradient of the classifier $\nabla_{z_n^t} \log p_\phi(c_n|z_n^t, t)$

is used to guide the denoising process towards an image with label c. The strength of the *guidance* is governed by a variable s. A linear combination of the denoising network output and the classifier output is used. Referencing back to the original example, where the denoising model was predicting the mean of a Gaussian distribution ($p_\theta(z_{t-1}|z_t) = \mathcal{N}(z_{t-1}|\mu(z_t, w, t), \beta_t \mathrm{I})$), the mean of the Gaussian is changed to the linear combination of the mean and the classifier as presented in Eq. 2.9.

$$p_\theta(z_{t-1}|z_t, c) = \mathcal{N}(z_{t-1}|\mu(z_t, w, t) + s\nabla_{z_t} \log p_\phi(c|z_t, t), \beta_t \mathrm{I}) \tag{2.9}$$

The limitation of the *classifier guidance* method is that it requires the training of an additional classifier, which must be able to classify differently noised data samples. Aside from the added complexity, concerns about this approach resembling gradient based adversarial attacks raise the question of whether they perform well only on classifier-based metrics, which are the ones used for evaluating the classifier guidance model in the original paper.

This motivated the ***classifier-free guidance*** method, presented by Ho and Salimans (2021). An elegant solution that does not require a second network to be trained. They make two changes to the original diffusion model by Ho et al. (2020): (i) they add a parameter to the denoising network (using the image example from before that is the $\mu(z_t, w, t)$) and they effectively train two models – one conditioned and one unconditioned. For the conditioned model, they pass the condition c as a parameter, and for the unconditioned model, they use the same network, but use a null token $\varnothing$ in place of c. However, as mentioned before, this is not enough, because the model might ignore the parameter. Therefore, (ii) they change the way the mean is calculated. They use a linear combination of the conditioned and unconditioned model result, as shown in Eq. 2.10, where w is the guidance parameter.

$$p_\theta(z_{t-1}|z_t, c) = \mathcal{N}(z_{t-1}|(1 + w)\mu(z_t, w, t, c) - w\,\mu(z_t, w, t), \beta_t \mathrm{I}) \tag{2.10}$$

Classifier-free diffusion guidance performs better than classifier guidance, particularly because the classifier in the classifier guidance, $p(c|x)$, can give insufficient weight (i.e. ignore) most of x as long as it outputs good predictions of c.

2.3.2 Diffusion Models for Graph Generation

Research on the use of diffusion models for graph-structured data can be characterised by two key aspects. The first aspect concerns the component of the graph that is diffused (i.e. noised) and subsequently generated. That component can either be the node attributes, the graph topology (represented by the adjacency matrix), or a combination of both. The second aspect is the type of

distribution used to noise the data, which can be either continuous (e.g. Gaussian) or discrete (e.g. Bernoulli).

The earliest approaches in this domain consider applying diffusion models to the adjacency matrix. Huang et al. (2022a) employ a diffusion model, that noises a binary symmetric adjacency matrix to an Erdős–Rényi random graph, with an edge probability of $p = 0.5$. The adjacency matrix is first linearly scaled from $[0, 1]$ to $[-1, 1]$, and then Gaussian noise is added to it such that at diffusion step T all values are set to 0.5. Therefore, this new adjacency matrix presents the probability of the edges existing. To create the graph topology from the new adjacency matrix, the edges are sampled using a Bernoulli distribution with the respective probabilities. A paper written at the same time by Jo et al. (2022) presents a diffusion model that models both the node features and the adjacency matrix through a system of stochastic differential equations. Their model has two diffusion processes that run in parallel, one for the node features and one for the adjacency matrix, both using Gaussian noise.

Two concurrent works, by Haefeli et al. (2022) and Vignac et al. (2023), are pivotal as they introduce discrete diffusion to graph data. That is a diffusion that uses discrete noise instead of continuous Gaussian noise and is first introduced by Austin et al. (2021). The model by Haefeli et al. (2022) is similar to the one by Huang et al. (2022a) in that it noises a discrete-valued adjacency matrix to an Erdős–Rényi random graph. However, the difference is that Haefeli et al. (2022) use a Categorical distribution to noise the matrix directly. Noising the data refers to changing the "category" of an element in the matrix. The "category" is the one-hot encoding of the matrix element; in a binary adjacency matrix the one-hot encodings of 0 and 1 are 01 and 10. At the start of the diffusion process, the probability of changing a category is 0, but as the time steps reach T, the probability becomes 0.5. Therefore, at time T the graph is an Erdős–Rényi random graph with probability $p = 0.5$. The model by Vignac et al. (2023) is similar in that nodes and edges are categorical. During diffusion their model progressively edits the graph by adding or removing edges and changing the categories of the nodes and edges, such that at time T the distribution over the node and edge categories is uniform. The latest state-of-the-art model for graph generation is the EDGE model, presented by Chen et al. (2023), which is a discrete diffusion model that focuses on adjacency matrix generation, and can scale to graphs with tens of thousands of nodes. This is the model on which we base the proposed node-guided topology reconstruction model for this thesis. The original EDGE model is described in detail in Section 2.4.

2.4 EDGE

In this section, the EDGE model by Chen et al. (2023) is presented in detail, because we use it to build our node-guided topology reconstruction diffusion model.

The EDGE model is a discrete-diffusion model that learns to reverse an edge removal process for homogeneous undirected graphs. It learns how to connect nodes that have only degrees as features, but the number of nodes and the degrees are not given by a modeller; they are sampled from a distribution created by the given dataset. The generation process can be viewed as a sampling of adjacency matrices. If $\mathcal{A}^N$ denotes the space of adjacency matrices of size N that are binary and symmetric, then the generative model defines a distribution over $\mathcal{A}^N$.

2.4.1 The Forward Process

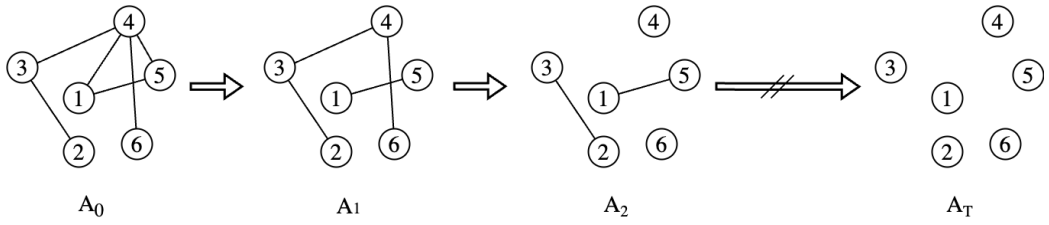


Figure 2.2: The forward process of the EDGE model

Looking at the forward process illustrated in Figure 2.2, A^0 denotes an adjacency matrix from the data, and the diffusion process defined by $q(A^t \mid A^{t-1})$ corrupts A^0 in T diffusion steps. q is a probability distribution over $\mathcal{A}^N$ and $A^1, ..., A^T$ are the latent variables. The forward process is a Markov process, formalised in Eq. 2.11

$$q(A^1, ..., A^T|A^0) = \prod_{t=1}^{T} q(A^t|A^{t-1}). \tag{2.11}$$

The authors use a Bernoulli distribution to model the forward transition and treat each edge independently as shown in Eq. 2.12. In the formula, $\mathcal{B}(x;\mu)$ denotes the Bernoulli distribution over x with probability μ. Bold parameters, $\mathcal{B}(\mathbf{x};\boldsymbol{\mu})$, are used to represent vectors/matrices. The rate at which the data is noised is governed by a noise scheduler β_t. In the formula, it determines the probability of resampling the entry of $A^t_{i,j}$ from a Bernoulli distribution with probability p instead of keeping it the same as $A^{t-1}_{i,j}$. Because the model is applied only to symmetric matrices, they model the upper triangle of the adjacency.

$$\begin{aligned} q(A^t|A^{t-1}) &= \prod_{i,j,i<j} \mathcal{B}(A^t_{i,j}; (1-\beta_t)A^{t-1}_{i,j} + \beta_t p) \\ &:= \mathcal{B}(\mathbf{A}^t; (1-\beta_t)\mathbf{A}^{t-1} + \beta_t p) \end{aligned} \tag{2.12}$$

Diffusion models are trained non-sequentially with respect to time. The diffusion step t is sampled for each element in the batch, thus sampling A^t from any time t directly from A^0 is required *for efficiency*. Therefore, the noise scheduler needs to be transformed, introducing two new variables $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_\tau^t \alpha_\tau$. As a result, sampling $q(A^t|A^0)$ is possible and defined in Eq 2.13.

$$q(A^t|A^0) = \mathcal{B}(\mathbf{A}^t; \bar{\alpha}_t \mathbf{A}^0 + (1-\bar{\alpha}_t)p) \tag{2.13}$$

The noise schedule is set in such a way that A^T is almost independent from A^0: $q(A^t|A^0) \approx p(A^T) \equiv \mathcal{B}(A^T; p)$. The authors set p equal to zero, so that at time T the graph has no edges. Lastly, to be able to compute the training loss, the posterior of the forward process is computed using the Bayesian theorem, resulting in Eq. 2.14

$$q(A^{t-1}|A^t, A^0) = \frac{q(A^t|A^{t-1})q(A^{t-1}|A^0)}{q(A^t|A^0)} \tag{2.14}$$

2.4.2 The Reverse Process

For the reverse process, an approximation of the reverse distribution $q(A^{t-1}|A^t)$ is learned by a deep neural network, which is denoted by $p_\theta(A^{t-1} \mid A^t)$, where θ are the model weights. The reverse distribution is modelled as a Bernoulli distribution, and the neural network predicts the probabilities of the edges existing: $\mu_\theta(\mathbf{A}^t, t)$. The reverse process is presented in Eq. 2.15. The process forms a Markov chain, and the joint distribution is presented in Eq. 2.16.

$$p_\theta(A^{t-1} \mid A^t) = \mathcal{B}(\mathbf{A}^{t-1} \mid \mu_\theta(\mathbf{A}^t, t)) \tag{2.15}$$

$$p_\theta(A^{0:T}) = p(A^T) \prod_{t=1}^{T} p_\theta(A^{t-1} \mid A^t) \tag{2.16}$$

The neural network modelling the reverse process is trained to maximise the variational lower bound of $\log p_\theta(A^0)$. Essentially, the goal of the model is to match the posterior of the forward transition. During generation, an empty graph is given, and the model *denoises* it iteratively using $p_\theta(A^{t-1} \mid A^t)$ to get A^0. In practice, the reverse process is a link predictor, which uses a graph attention network for node embedding generation. The training procedure is explained in Section 2.4.4.

2.4.3 Addition of Active Nodes

The link predictor used in the reverse process has to compute the probability for every edge between every two nodes; however, as the graph grows, this becomes computationally expensive. The solution of the authors is to predict edges only between a subset of the nodes. If the probability used for the Bernoulli of the forward process p is set to zero, the forward process only removes edges, and the degrees of nodes are non-increasing for any forward transition. To combat the scalability issue, the authors of the paper introduce the notion of *active nodes*: a node is active at time t if there was a change in its degree between time steps $t-1$ and t, where d^t denotes the degree of a given node at time step t. The number of active nodes per time step depends on the number of removed edges during the forward process, which is governed by the noise schedule β. During the reverse process, the model predicts edges only between active nodes, decreasing the computational cost.

Active nodes are treated as latent variables $s^{1:T}$, where $s^t := \mathbb{1}\left[d^{t-1} \neq d^t\right]$ is a binary vector indicating which nodes are active between time steps $t-1$ and t. Adding the active nodes to the forward process does not change it, because the active nodes are determined by A^{t-1} and A^t, shown in Eq. 2.17. The reverse process is presented in Eq. 2.18, in which both $p_\theta(A^{t-1}|A^t, s^t)$ and $p_\theta(s^t|A^t)$ are learnable distribution parametrized by θ.

$$q(A^{1:T}, s^{1:T}|A^0) = \prod_{t=1}^{T} q(A^t|A^{t-1}) q(s^t|A^{t-1}, A^t) \tag{2.17}$$

$$p_\theta(A^{0:T}, s^{1:T}) = p(A^T) \prod_{t=1}^{T} p_\theta(A^{t-1}|A^t, s^t) p_\theta(s^t|A^t) \tag{2.18}$$

2.4.4 Loss Function

The model parameters θ are optimised by maximising the variational lower bound of $\log p_\theta(A^0)$ presented in Eq. 2.19. The full derivation of the formula can be found in Appendix A.

$$\begin{aligned}
&\mathcal{L}(A^0;\theta) = \log p_\theta(A^0) \\
&= \mathbb{E}_q \Big[\log \frac{p(A^T)}{q(A^T|A^0)} + \underbrace{\log p_\theta(A^0|A^1, s^1)}_{\text{reconstruction term } \mathcal{L}_{rec}} \\
&\quad + \sum_{t=2}^{T} \underbrace{\log \frac{p_\theta(A^{t-1}|A^t, s^t)}{q(A^{t-1}|A^t, s^t, A^0)}}_{\text{edge prediction term } \mathcal{L}_{edge}(t)} + \sum_{t=1}^{T} \underbrace{\log \frac{p_\theta(s^t|A^t)}{q(s^t|A^t, A^0)}}_{\text{node selection term } \mathcal{L}_{node}(t)} \Big]
\end{aligned} \tag{2.19}$$

In the equation, the first term has no learnable parameters. The second term measures the reconstruction likelihood. Regarding the third term, the posterior $q(A^{t-1}|A^t, s^t, A^0)$ is hard to compute and the term has no closed-form. Since the entropy of the posterior is a constant, the authors suggest the use of Monte Carlo estimates to optimise the cross-entropy term of $\mathcal{L}_{edge}(t)$. Looking at the last term, $q(s^t|A^t, A^0)$ has closed-form expression, but to present it, first the posterior of the node degree distribution has to be introduced – Eq. 2.20 , where $Bin(k, n, p)$ is binomial distribution parametrized by n and p.

$$\begin{aligned} q(d^{t-1}|A^t, A^0) &= q(d^{t-1}|d^t, d^0) = \prod_{t=1}^{N} q(d_i^{t-1}|d_i^t, d_i^0) \\ q(d_i^{t-1}|d_i^t, d_i^t) &= Bin(k = \Delta_i^t, n = \Delta_i^0, p = \gamma_t) \\ \Delta_i^t &= d_i^{t-1} - d_i^t \\ \Delta_i^0 &= d_i^0 - d^t \\ \gamma_t &= \frac{\beta_t \bar{\alpha}_t - 1}{1 - \bar{\alpha}_t} \end{aligned} \tag{2.20}$$

Since nodes are active when the node degrees are not equal, $s_i^t = \mathbb{1}\left[d_i^{t-1} \neq d_i^t\right]$, the probability $q(s_i^t = 1|d_i^t, d_i^0)$ is $1 - q(d_i^{t-1} = d_i^t|d_i^t, d_i^0)$. With that, the closed-form can be computed and is presented in Eq. 2.21. Therefore, the last term of the loss is a comparison between Bernoulli distributions.

$$\begin{aligned} q(s^t|d^t, d^0) &= \prod_{i=1}^{N} q(s_i^t|d_i^t, d_i^0) \\ q(s_i^t|d_i^t, d_i^0) &= \mathcal{B}(s_i^t; 1 - (1 - \gamma_t)^{\Delta_i^0}) \end{aligned} \tag{2.21}$$

2.4.5 Degree Guidance

Chen et al. (2023) argue that node degrees are strongly correlated to a graph's other statistics, thus a diffusion model that incorporates them would generate graphs similar to the training data. They model the degrees of graphs as random variables. Eq. 2.22 presents how the forward process is changed, while Eq. 2.23 shows the reverse process. The forward process is not changed because the probability $q(d^0|A^0) = 1$ because the degrees are determined by the initial adjacency matrix A^0. In the reverse process, the model first samples the node degrees d^0 from a probability distribution $p_\theta(d^0)$, and then it generates a graph guided by the sampled degrees. It is important to note, that although they call the addition of the nodes degreed to the model "degree guidance", this guidance

is not in any way similar to the guidance methods previously discussed in Section 2.3.1. They pass the node degrees as a parameter to the network used for link prediction during the denoising step, but they do not train the model with the null token in place of the degrees, and they do not use a linear combination of conditioned and unconditioned link prediction. Instead, they use the node degrees to assist with the sampling of the active nodes.

$$q(A^{1:T}|A^0) = q(A^{1:T}|A^0)q(d^0|A^0) \tag{2.22}$$

$$p_\theta(A^{0:T}, s^{1:T}, d^0) = p_\theta(d^0)p_\theta(A^{0:T}, s^{1:T}|d^0) \tag{2.23}$$

2.4.6 Loss Function of Degree Guided Variation

The optimization of the node degree model $p_\theta(d^0)$ is separate from the graph denoising model $p_\theta(A^{0:T}, s^{0:T} \mid d^0)$. Therefore the training objective (Eq. 2.24) is split in two, with $\mathcal{L}(d^0;\theta)$ treated as a sequence modelling task – fitting $p_\theta(d^0)$ to the data's node degree distribution $p_{data}(d^0)$.

$$\mathcal{L}(A^0, d^0;\theta) = \mathbb{E}_q \Big[\underbrace{\log p_\theta(d^0)}_{\mathcal{L}(d^0;\theta)} + \underbrace{\log \frac{p_\theta(A^{0:T}, s^{1:T}|d^0)}{q(A^{1:T}, s^{1:T}|A^0)}}_{\mathcal{L}(A^0|d^0;\theta)} \Big] \tag{2.24}$$

The $\mathcal{L}(A^0|d^0;\theta)$ is decomposed as in Eq. 2.19, but all the terms related to the graph generative model are conditioned on d^0. Because the reverse process is conditioned on d^0, predicting the active nodes for the reverse process can be made equivalent to the forward one, thus, the authors set $p_\theta(s^t|A^t, d^0) := q(s^t|A^t, A^0)$ resulting in KL divergence of $\mathcal{L}_{node}(t) = 0$ for all t. With this, the simplified loss is presented in Eq. 2.25.

$$\mathcal{L}(A^0|d^0;\theta) = \mathbb{E}_q \Big[\log \frac{p(A^T)}{q(A^T|A^0)} + \underbrace{\log p_\theta(A^0|A^1, s^1, d^0)}_{\text{reconstruction term } \mathcal{L}_{rec}} + \sum_{t=2}^{T} \underbrace{\log \frac{p_\theta(A^{t-1}|A^t, s^t, d^0)}{q(A^{t-1}|A^t, s^t, A^0)}}_{\text{edge prediction term } \mathcal{L}_{edge}(t)} \Big] \tag{2.25}$$

Although not evident from the objective function above, the node degrees play a crucial role in the selection of active nodes – only nodes with a degree below the specified degree d^0 may be selected as active. Equation 2.26 shows how the final reverse transition is formulated. The model predicts edges only between nodes that are both active. Therefore, only when both nodes i and j are active $s^t_{ij} = s^t_i s^t_j$, the model uses the output of the graph neural network μ_θ to determine if there is an edge, otherwise the value of the adjacency matrix stays the same.

$$
\begin{aligned}
p_\theta(A^{t-1}|A^t, s^t, d^0) &= \prod_{i,j;i<j} p_\theta(A^{t-1}_{i,j}|A^t_{i,j}, s^t_{i,j}, d^0) \\
p_\theta(A^{t-1}_{i,j}|A^t_{i,j}, s^t_{i,j}, d^0) &= \mathcal{B}(A^{t-1}_{i,j}|\, s^t_{i,j}\, \mu_\theta(A^t, s^t, d^0, t) \;+\; (1 - s^t_{i,j})\, A^t_{i,j}) \\
s^t_{ij} &= s^t_i s^t_j
\end{aligned}
\tag{2.26}
$$

2.4.7 EDGE Implementation

The denoising network $\mu_\theta(A^t, s^t, d^0, t)$ is a link predictor based on a graph neural network that incorporates attention. The graph attention neural network takes A^t, d^0, s^t, and t as input and computes node representations $Z^t \in \mathbb{R}^{n \times d_h}$ where n is the number of nodes and d_h is the hidden dimensions, a hyperparameter. There are two other hyperparameters – the depth of the neural network, that is the number of times the aggregation of information from neighbouring nodes is repeated, as well as the number of attention heads each layer has. Then, using the embedding of the last hidden layer for the active nodes $Z^t[s^t]$, the network predicts the probability of a link (edge) between the active nodes s^t. Once that probability is obtained, it is used for the Bernoulli sampling.

2.5 Anomaly Detection

An anomaly is an observation – or, in graphs, a node, edge, or subgraph – which deviates from the rest of the data, and raises suspicions whether it is generated by the same underlying probability distribution (Hawkins, 1980; Barnett and Lewis, 1994). The identification of those instances is called anomaly detection, and it is used in various fields from medicine to financial fraud.

From the supervised methods for classification and pattern recognition, only those that can work effectively with highly unbalanced data can be used for anomaly detection. As an example, Support Vector Machines (Schölkopf and Smola, 2002) and deep neural networks would perform better than decision trees. However, supervised methods are not commonly used, because they require data labelled with "normal"/"anomalous", which is unavailable in most cases. The commonly used methods for anomaly detection are unsupervised approaches, which do not require any labelling, except for their evaluation. Examples of such algorithms are Isolation Forest, k-nearest-neighbour, and local outlier factor (Goldstein and Uchida, 2016). The first algorithm is further discussed in detail, because it serves as a classical baseline in our experiments (Section 4).

2.5.1 Isolation Forest

The Isolation Forest algorithm (Liu et al., 2008) relies on the assumption that anomalies are "few and different", meaning that anomalous instances are the minority, consisting of fewer instances, and that they have attribute values that are different from those of normal instances. The proposed method is novel in that it does not model instance normality, but explicitly isolates anomalies. The model works by creating multiple data-induced random proper binary trees, where each node in the tree has exactly zero or two daughter nodes. The trees partition (sub-sampled) instances recursively until a predetermined tree height is reached, or all instances have the same attribute values. The partitioning is random and is created in two steps: (i) an attribute is selected at random, and (ii) a split value between the minimum and the maximum values of the selected attribute is randomly selected. The anomaly score for each instance is calculated as a function of the average path length to the instance over all generated trees. A path length is defined as the number of tree edges between the root of the binary tree and the leaf where the instance resides. Anomalous instances are more susceptible to isolation and hence have short path lengths. An example is illustrated in Figure 2.3, where given a Gaussian distribution, a normal point x_i requires twelve random partitions to be isolated, whereas an anomalous point x_0 requires only four partitions.

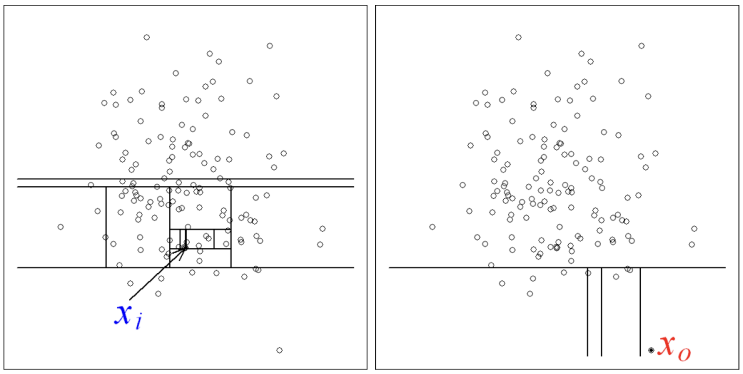


Figure 2.3: Isolation Forest algorithm isolating anomalous instance x_0 in less splits than isolating normal instance x_i. Figure adopted from Liu et al. (2008)

To obtain an anomaly score for an instance x from the path length $h(x)$, we need a normalisation value, however, we cannot use the maximum possible height of a tree, nor the average height of the tree. The reason is that given a dataset with n instances, the maximum possible height grows in the order of n, whereas the average height grows in the order of $\log n$. Isolation trees and binary

search trees have equivalent structures, therefore Liu et al. (2008) borrow formulation from binary search trees to define the average path length of a tree as presented in Eq. 2.27, where $H(i)$ is the harmonic number and it can be estimated by $ln(i) + 0.5772156649$ (Euler's constant).

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n} \tag{2.27}$$

Therefore, the anomaly score s of an instance x is formulated as presented in Eq. 2.28, where $E(h(x))$ is the average path length to instance x from a collection of isolation trees. The score s is monotonic to $h(x)$. If an instance has a score $s(x)$ very close to 1, then it is definitely an anomaly. When the score is significantly smaller than 0.5, then the instance is regarded as normal. If all instances in the dataset have a score of around 0.5, then there are no distinct anomalies.

$$s(x,n) = 2^{-\frac{E(h(x))}{c(n)}} \tag{2.28}$$

The Isolation Forest algorithm has two parameters – (i) the sub-sampling size and (ii) the number of trees. The sub-sampling size is the number of samples used to train a single tree. The authors found that using around 2^8 number of samples is enough to create a deep enough tree without overfitting to the data. They also recommend using around 100 trees. The maximum tree height can be calculated from the subsampling size in the following way: height = ceiling($\log_2$ subsampling size).

The Isolation Forest algorithm can be used for graph anomaly detection through the use of graph representations, discussed in Section 2.2. If the task is node anomaly detection, and the graph does not have node attributes, or the attributes are insufficient by themselves to detect anomalies, using any of the discussed algorithms for embedding generation, such as graph neural network, could be used in place of, or in addition to, the node attributes. The approach would be similar if the task is edge anomaly detection, or graph anomaly detection.

2.5.2 Graph Anomaly Detection using Diffusion Models

Aside from the conventional machine learning methods discussed above, new research in the area of anomaly detection is trying to adapt generative models. The core idea behind using generative models for anomaly detection lies in the assumption that generative models, given enough data, learn to generate normal data. Therefore, any deviation from what they have generated is considered anomalous. Using generative models for anomaly detection has already been explored for different

applications of from medical anomaly detection (Wolleb et al., 2022) to time series anomaly detection for manufacturing and the IT infrastructure (Pintilie et al., 2023).

Although many diffusion models exist for graph generation, as presented in Section 2.3.2, using them for anomaly detection is not a straightforward process, as they model the global properties of the graphs. In the context of graphs, for the detection of anomalies using a generative model, it is required that one of the graph's components (either the node attributes or the graph topology) is used for contextualising and guiding the generation process.

There is one published model inline with this idea by Li et al. (2023). They propose an anti-money laundering framework that is based on a diffusion model with classifier guidance. They diffuse and reconstruct the node attributes towards non-money laundering behaviour. For the diffusion model, they use Gaussian noise and train the classifier on pairs of (noised) graphs with money-laundering labels. During inference, if there is a big difference between the original and the reconstructed node attributes, they conclude that the original exhibits money-laundering behaviour. Their assumption is that the node attributes are determined by their connections.

There are three differences between their model and the one we propose : (i) our model lies under the assumption that the node connections are dependent on the node attributes, therefore instead of keeping the topology of the graph and diffusing the node attributes, we do the opposite; (ii) their model is supervised, ours is unsupervised; (iii) their model requires training an additional classifier because they use classifier guidance for the diffusion, whereas we use classifier-free guidance.

2.6 Link Prediction

Graph anomaly detection and link predictions are two closely related tasks. Link prediction is defined as the problem of determining whether an edge exists between two given nodes. Our proposed method uses a link predictor at its core, therefore, exploring the literature in this domain is essential.

Similar to graph anomaly detection, different supervised and unsupervised methods can be applied to the task, with the use of graph neural networks being central in a majority of them, as they are used for embedding generation. However, not many of the proposed methods make use of generative models (Arrar et al., 2024). Notable is the work of Wang et al. (2020), in which they use generative adversarial network for adjacency matrix generation. They suggest training the generative model on samples, created by slightly perturbing the original adjacency matrix, therein, resulting in a model able to generate the missing edges.

The most recent work utilising generative models for link prediction is by Li et al. (2024). They model the probability of a central edge of a subgraph existing using Bayes' theorem and two diffusion models, both conditioned on the fact that the central edge exists. One diffusion model for the probability of the adjacency matrix and another for the node features. Eq.2.29 presents the formulation they are using. In the formula, A is the adjacency matrix and X is the node feature matrix of the subgraph induced by the two-hop neighbours of the nodes on each side of the edge e_{uv}. $p(y)$ is the prior distribution of node-pairs' connection status over the graph. $p(A|y)$ denotes the graph structure probability given that edge e_{uv} exists, and $p(X|A, y)$ represents the node feature probability that is conditioned on the observed structure and connection status.

$$p(y|A, X) = \frac{p(X|A, y) \cdot p(A|y) \cdot p(y)}{\sum_{c \in \{0,1\}} p(A, X|y = c) \cdot p(y = c)} \tag{2.29}$$

For $p(A|y)$ they use the model proposed by Vignac et al. (2023), and for $p(X|A, y)$ they follow the model presented by Ho et al. (2020) and use a graph neural network to model the noise.

The literature review presented a potentially significant research gap – diffusion models for graph topology generation have not been explored as a tool for unsupervised graph anomaly detection. The probable reason is that there is no diffusion model designed to reconstruct node connections for a given set of nodes and their attributes while obeying local graph properties. Therefore, we propose a classifier-free node-guided topology reconstruction model that uses discrete diffusion on the adjacency matrix to generate edges between attributed nodes with given node degrees. We further use this model to compare the original node topology with the reconstructed one to determine if there are any anomalies in the graph.

CHAPTER

THREE

THE ALLIGATOR MODEL

This work proposes the alliGATOR model – an unsupervised graph anomaly detection through diffusion based topology reconstruction. The novelty of the proposed approach is that anomalies are discovered by comparing the original graph topology with a reconstructed one. The model has three nested components. The outermost one is an anomaly score component, which depends on a diffusion model that generates graph edges using a link predictor, which is the innermost component. This model can be used for both node and edge anomaly detection. The difference between the two modes is the preprocessing of the input graph and the anomaly score formula. The second contribution of this work is the node-guided diffusion model for topology reconstruction, designed to model the local, micro-level properties of nodes and edges in the graph. The diffusion model is built on the EDGE architecture by Chen et al. (2023), as described in Section 2.4.

The Chapter is divided into model motivation (Section 3.1), model formulation (Section 3.2), model implementation (Section 3.3), complexity analysis of the model (Section 3.4), and hyperparameter setting (Section 3.5)

3.1 Model Motivation

Detecting anomalies has proven to be an important, but challenging task. This is predominantly due to the limited availability of labelled data and the heavily unbalanced nature of the anomaly datasets; by definition, anomalous instances are significantly less than normal ones. As per the aim of this thesis, we propose an unsupervised approach, that not only accommodates, but potentially leverages this imbalance for improved anomaly detection performance.

Generative models are particularly well-suited for this task and have been previously proposed for anomaly detection across different data modalities. Their appropriateness for the task is rooted in two interconnected assumptions. First, modelling the underlying distribution of the normal data is easier compared to modelling the distribution of the anomalous instances. Second, due to the limited availability of anomalous instances in the input data, generative models do not have sufficient information to learn their distribution. The above assumptions align closely with those of the Isolation Forest, which presume that anomalies are "few and different". Anomalies are detected by measuring the discrepancy between the input data and the data generated by the model, where significant deviations suggest the presence of anomalous instances.

We are interested in graph-structured data, which can represent financial transactions, biological molecules, social relations, and other important aspects of our lives. While there are many graph generative models, as presented in Section 2.1.1 and Section 2.3.2, their application to graph anomaly detection is limited and underexplored. The key reason for this is that existing models are designed to generate realistic synthetic graphs. These models focus on the global properties of the graphs, such as the number of nodes and edges, the degree distribution, the diameter of the graph, the connectivity, the centrality, and the clustering coefficient.

The detection of node-, edge-, and subgraph-level anomalies requires a model that can capture the local properties of the graph, such as the connectivity patterns of individual nodes. Therefore, the generation process must be contextualised. To model a node's attributes using a generative model, the process must be conditioned on the surrounding graph topology. Conversely, to model the connectivity pattern of a node, the generation process must be conditioned on the node's attributes. The assumptions underlying those two approaches are complementary. The first approach assumes that node attributes are dependent on the connections, while the second approach assumes that the node connections are inherent to the node attributes.

With the above in mind, we propose a model that detects node and edge anomalies using a diffusion model, designed to reconstruct the topology of a graph under the guidance of the node attributes. The assumption that node connections are dependent on the node attributes aligns with the financial context of our research. We call our model the alliGATOR model : ***G****raph* ***A****nomaly Detection through diffusion based* ***To****pology* ***R****econstruction.* The model uses a node-guided diffusion model to reconstruct the topology, which is our second contribution.

3.2 Model Formulation

The input to the alliGATOR model is a graph $G(V, E, X)$, with V the set of nodes, E the set of edges, and X the node attributes. The output is an anomaly score for each node or edge, depending on whether the task is respectively node or edge anomaly detection. The first step of the algorithm is to create a subgraph for each instance of interest. This subgraph is then stripped of its edges and fed into a trained node-guided diffusion model for topology reconstruction, the output of this model are the reconstructed edges. The topology reconstruction process is repeated several times, and probabilities of each edge are obtained from the frequencies of occurrence in the reconstructed topologies. The last step of the alliGATOR model is the comparison between the initial subgraph edges and the reconstructed ones. This comparison results in an anomaly score for the instance for which the subgraph was created around. The alliGATOR model is visualised in Figure 3.1.

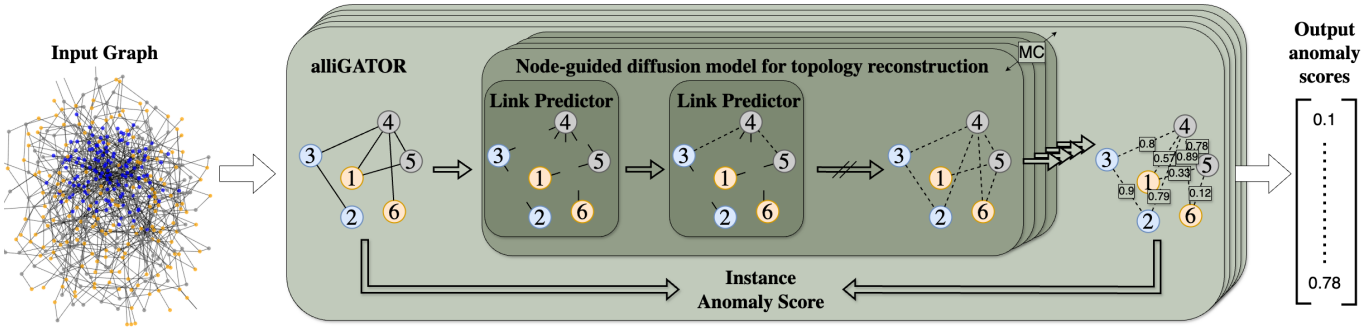


Figure 3.1: Anomaly score generation using the alliGATOR model

3.2.1 The Node-Guided Diffusion for Topology Reconstruction

We decided to base our node-guided topology reconstructor on the model by Chen et al. (2023), described in Section 2.4. There are a few reasons we settle on this specific model. Firstly, although their model is for graph generation, meaning both nodes and edges are being generated, their focus is on the edge generation, and they diffuse and reconstruct the adjacency matrix. In their model, the number of nodes with their degrees are generated from another network, which we are not going to use. The second reason we chose the EDGE model is because they use a discrete diffusion for the adjacency matrix, which, as previously discussed in Section 2.3, is the preferred way, because the adjacency matrix has discrete values. The third reason behind our choice is that the EDGE model

can scale up to graphs with tens of thousands of nodes. This is possible because of the addition of "active nodes". Instead of predicting edges between all nodes in the graph, the model predicts edges only between a subset, therefore, significantly reducing the computational complexity. Lastly, the authors made their implementation of the EDGE model publicly available [1] . The implementation of our alliGATOR model is also publicly available [2].

To adapt the EDGE model from a graph generator to a topology reconstructor, we first considered the ways of incorporating the node attributes into the denoising network. We settled on using classifier-free guidance as presented in the paper by Ho and Salimans (2021). The reasons behind this decision are twofold. Firstly, in contrast to classifier guidance, we do not need to train an additional model that gives the probability of the node attributes given the adjacency matrix. Secondly, implementing classifier-free guidance is easier because it requires extending the neural network that models the *reverse process* with one parameter, i.e. the condition. In our case, that requires extending the link predictor model to take node attributes into account during the message passing.

The Forward Process The addition of classifier-free node-guidance does not change the forward process, because it is an edge removal process, thus, it focuses only on the adjacency matrix. The forward process remains the same as in the EDGE model, described in Section 2.4.1. Equation 2.12 presents the forward transition between two consecutive states from Section 2.4.1. The forward transition is modelled by a Bernoulli distribution. Every element in the adjacency matrix (i.e. every edge) is sampled individually. The noise schedule β governs the rate at which edges are removed. With a probability of $(1-\beta_t)$, the edge is kept as in state A^{t-1}, with a probability of β_t the edge is resampled from a Bernoulli with probability p. As the diffusion step t increases, β grows, therefore, towards the last step of the diffusion, the probability of each existing would be p. p to set to zero, because this makes the graph sparse and decreases the computational complexity. The forward process is illustrated in Figure 3.2.

$$
\begin{aligned}
q(A^t|A^{t-1}) &= \prod_{i,j,i<j} \mathcal{B}(A^t_{i,j}; (1-\beta_t)A^{t-1}_{i,j} + \beta_t p) \\
&:= \mathcal{B}(\mathbf{A}^t; (1-\beta_t)\mathbf{A}^{t-1} + \beta_t p)
\end{aligned}
\qquad \text{(2.12 copied from page 16)}
$$

[1] Original implementation of EDGE :https://github.com/tufts-ml/graph-generation-EDGE

[2] alliGATOR implementation available at https://github.com/YuliaTerzieva/MScThesis

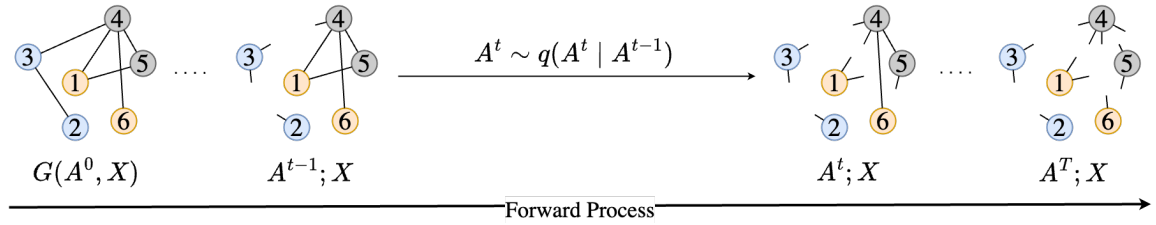


Figure 3.2: Forward process of the node-guided diffusion model for topology reconstruction

The Reverse Process The reverse process is modelled by a Bernoulli distribution with a neural network predicting the edge probabilities, however, the difference is that the neural network receives one more parameter – the node attributes, denoted by X. The previous formulation of the reverse process is presented by Eq. 2.26. The new reverse process, which includes the node attributes, is presented in Eq. 3.1. The only difference between the original reverse process from the EDGE model, which includes the node degrees and the active nodes, and the node-guided reverse process is the addition of X to the network that predicts the edge probabilities between nodes.

$$
\begin{aligned}
p_\theta(A^{t-1}|A^t, s^t, d^0) &= \prod_{i,j;i<j} p_\theta(A^{t-1}_{i,j}|A^t_{i,j}, s^t_{i,j}, d^0) \\
p_\theta(A^{t-1}_{i,j}|A^t_{i,j}, s^t_{i,j}, d^0) &= \mathcal{B}(A^{t-1}_{i,j}|\ s^t_{i,j}\ \mu_\theta(A^t, s^t, d^0, t)\ +\ (1 - s^t_{i,j})\ A^t_{i,j}) \\
s^t_{ij} &= s^t_i s^t_j
\end{aligned}
\qquad \text{(2.26 copied from page 20)}
$$

$$
\begin{aligned}
p_\theta(A^{t-1}|A^t, s^t, d^0, X) &= \prod_{i,j;i<j} p_\theta(A^{t-1}_{i,j}|A^t_{i,j}, s^t_{i,j}, d^0, X) \\
p_\theta(A^{t-1}_{i,j}|A^t_{i,j}, s^t_{i,j}, d^0, X) &= \mathcal{B}(A^{t-1}_{i,j}|\ s^t_{i,j}\ \mu_\theta(A^t, s^t, d^0, X, t)\ +\ (1 - s^t_{i,j})\ A^t_{i,j}) \\
s^t_{ij} &= s^t_i s^t_j
\end{aligned}
\qquad (3.1)
$$

Adapting the EDGE model from graph generator to topology reconstructor means that we no longer need a separate model for the node degrees, because we provide the nodes, with their attributes and degrees, to the denoising model. Previously, Equations 2.24 and 2.25 presented the training objective for the EDGE model. We substitute both Equations with the revised loss – Eq. 3.2, where X denotes the node attributes. There is no closed-form computation of the expectation in Eq. 3.2, therefore, we optimise the model parameters using Monte Carlo estimates, following the formula from Eq. 3.3.

$$\mathcal{L}(A^0, d^0; \theta) = \mathbb{E}_q \Big[\underbrace{\log p_\theta(d^0)}_{\mathcal{L}(d^0;\theta)} + \underbrace{\log \frac{p_\theta(A^{0:T}, s^{1:T} | d^0)}{q(A^{1:T}, s^{1:T} | A^0)}}_{\mathcal{L}(A^0|d^0;\theta)} \Big] \qquad \text{(2.24 copied from page 19)}$$

$$\mathcal{L}(A^0|d^0;\theta) = \mathbb{E}_q \Big[\log \frac{p(A^T)}{q(A^T|A^0)} + \underbrace{\log p_\theta(A^0|A^1, s^1, d^0)}_{\text{reconstruction term } \mathcal{L}_{rec}} + \sum_{t=2}^{T} \underbrace{\log \frac{p_\theta(A^{t-1}|A^t, s^t, d^0)}{q(A^{t-1}|A^t, s^t, A^0)}}_{\text{edge prediction term } \mathcal{L}_{edge}(t)} \Big] \quad \text{(2.25 copied)}$$

$$\mathcal{L}(A^0|d^0, X;\theta) = \mathbb{E}_q \Big[\log \frac{p(A^T)}{q(A^T|A^0)} + \underbrace{\log p_\theta(A^0|A^1, s^1, d^0, X)}_{\text{reconstruction term } \mathcal{L}_{rec}} + \sum_{t=2}^{T} \underbrace{\log \frac{p_\theta(A^{t-1}|A^t, s^t, d^0, X)}{q(A^{t-1}|A^t, s^t, A^0)}}_{\text{edge prediction term } \mathcal{L}_{edge}(t)} \Big] \quad (3.2)$$

$$\nabla_\theta \mathcal{L}(A^0|d^0, X;\theta) = \mathbb{E}_{t\sim\mathcal{N}[1,T]} \Big[\mathbb{E}_{q(A^{t-1}|A^0)} \mathbb{E}_{q(A^t|A^{t-1})} \mathbb{E}_{q(s^t|A^{t-1},A^t)} \big[\log p_\theta(A^{t-1}|A^t, s^t, d^0, X) \big] \Big] \quad (3.3)$$

However, as noted by Bishop and Bishop (2023), passing the node attributes as a parameter is not enough, because the network might give insufficient weight or even ignore them. Thus, we train the denoising network *without* node attributes (i.e. with a null token in their place) $p_{uncon}\%$ of the time and *with* the node attributes otherwise. During generation, a linear combination of both, conditioned and unconditioned, link predictor outputs are used for the calculation of the edge probabilities used for the Bernoulli sampling, shown in Eq. 3.4. The conditioned model is $\mu_\theta(A^t, s^t, d^0, X, t)$, the unconditioned model is $\mu_\theta(A^t, s^t, d^0, t)$, and the guidance strength is w, it determines how the results are combined. We only calculate the edge probabilities for active edges s^t_{tj}, which are edges between two active nodes s^t_t and s^t_j. Tilda is used to show that the equation is used for generation only, while Eq. 3.1 is used for training. The generation process is presented in Figure 3.3. The figure must be read from right to left, as it presents the reverse process of the diffusion model. In the visualisation, the probabilities the link predictor outputs are marked with p for the conditioned and p' for the unconditioned.

$$\begin{aligned}
\tilde{\mu}_\theta(A^t, s^t, d^0, X, t) &= (1+w)\ \ \mu_\theta(A^t, s^t, d^0, X, t) - w\ \ \mu_\theta(A^t, s^t, d^0, t) \\
\tilde{p}_\theta(A^{t-1}|A^t, s^t, d^0, X) &= \prod_{i,j;i<j} \tilde{p}_\theta(A^{t-1}_{i,j}|A^t_{i,j}, s^t_{i,j}, d^0, X) \\
\tilde{p}_\theta(A^{t-1}_{i,j}|A^t_{i,j}, s^t_{i,j}, d^0, X) &= \mathcal{B}(A^{t-1}_{i,j}|\ s^t_{i,j}\ \tilde{\mu}_\theta(A^t, s^t, d^0, X, t)\ +\ (1 - s^t_{i,j})\ A^t_{i,j}) \\
s^t_{ij} &= s^t_i s^t_j
\end{aligned} \quad (3.4)$$

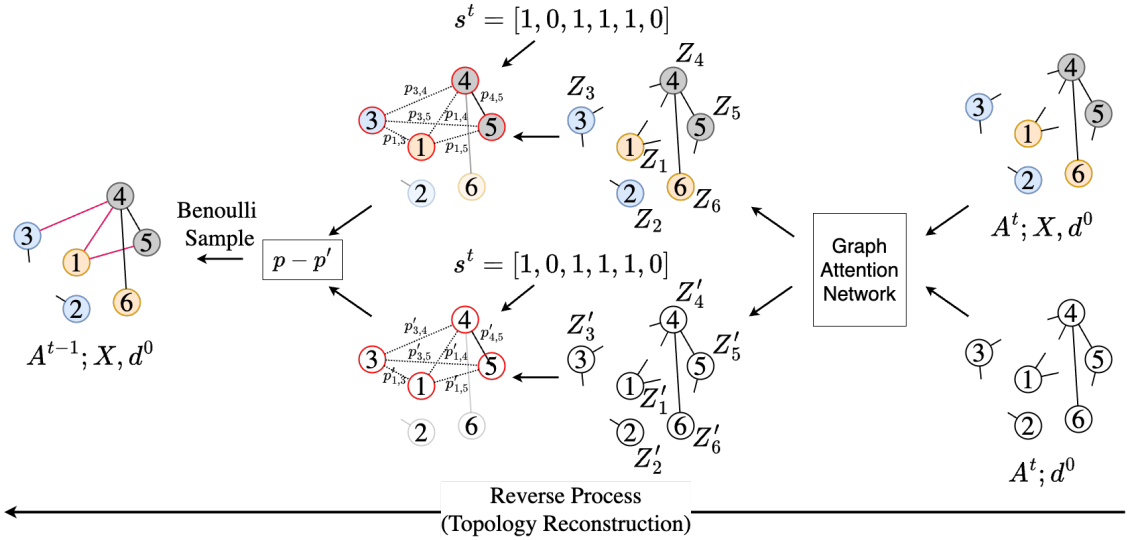


Figure 3.3: Reverse process of the node-guided diffusion model for topology reconstruction

The pseudo-code for training the node-guided diffusion model for topology reconstruction is provided in Algorithms 3.1. The input includes a set of training graphs $\mathcal{G} = \{G_1, G_2, ..., G_N\}$, where each graph has a binary symmetric adjacency matrix A_i and a matrix with node attributes X_i: $G_i = (A_i, X_i)$. The rest of the input includes the number of diffusion steps T, the forward transition distributions $q(A^t|A^{t-1})$ and $q(A^t|A^0)$, and the probability of unconditional training p_{uncon}. The following few steps are repeated until convergence: line 2, a graph is selected; on line 3 with probability p_{uncon} all the node attributes are set to the null token, otherwise kept the same. Line 4, the node degrees are extracted from the adjacency matrix A_i. Diffusion models are trained non-sequentially, therefore a time step needs to be sampled for every instance as presented on line 5 – a time step t is sampled from a uniform distribution between 0 and the diffusion steps T. For sampling the adjacency matrix $t-1$ is required as well. Line 6, given that we have A^0 we use the forward Bernoulli distribution $q(A^{t-1}|A^0)$, as presented in Eq. 2.13, to sample the adjacency matrix at time $t-1$: A^{t-1}. Line 7, given the newly sampled A^{t-1}, we can use the Bernoulli distribution $q(A^t|A^{t-1})$ as described in Eq. 2.12 to sample A^t. Line 8, given the adjacency matrices A^t and A^{t-1}, we have the node degrees for both time steps d^t and d^{t-1}, therefore we get the active nodes s^t by looking if a node has a change in node degrees (d^t vs d^{t-1}). Line 9, we perform gradient descent over the model parameters θ with gradient $\nabla_\theta \log p_\theta(A^{t-1}|A^t, s^t, d^0, X)$, for $p_\theta(A^{t-1}|A^t, s^t, d^0, X)$ presented in Eq. 3.1.

Algorithm 3.1: Training of node-guided diffusion model for topology reconstruction

Input: Graph set $\mathcal{G} = \{G_1, G_2, ..., G_N\}$, $G_i = (A_i, X_i)$, diffusion steps T, forward transition distributions $q(A^t|A^{t-1})$ and $q(A^t|A^0)$, probability of unconditional training p_{uncon}

Output: Denoising network $p_\theta(A^{t-1}|A^t, s^t, d^0, X)$

1 **repeat**
2 Draw $(A^0, X) \sim \mathcal{G}$
3 X $\leftarrow \varnothing$ with probability p_{uncon}
4 Compute node degree $d^0 = deg(A^0)$
5 Sample $t \sim \mathcal{U}[1:T]$, have $t-1$ as well
6 Draw $A^{t-1} \sim q(A^{t-1}|A^0)$
7 Draw $A^t \sim q(A^t|A^{t-1})$
8 Obtain active nodes s^t from A^{t-1} and A^t
9 Take gradient step on $\nabla_\theta \log p_\theta(A^{t-1}|A^t, s^t, d^0, X)$
10 **until** *convergence*;

Algorithm 3.2 presents the pseudo-code of the topology reconstruction process. Given a graph $G(A_{OG}, X)$, for which we want to reconstruct the edges, the other required inputs are the diffusion steps T, the active nodes distribution $q(s^t|d^t, d^0)$, the guidance strength w, and the denoising distribution $\tilde{p}_\theta(A^{t-1}|A^t, s^t, d^0, X)$. The topology reconstruction is initialised by the extraction of the node degrees of the original graph d^0 from the adjacency matrix A_{OG}. Then a matrix with the same shape as the original adjacency matrix is created A^T, but all the values are set to zero. Then, the following two steps are repeated T times generating the topology, where the variable t goes from T to 1, and on each step t the model generates the adjacency matrix for A^{t-1} :

line 4 Using the last created adjacency matrix A^t, the node degrees $d^t = deg(A^t)$ are extracted and using the distribution $q(s^t|d^t, d^0)$ as described in Eq. 2.20 and 2.21 the active nodes for time step t are sampled.

line 5 Using the last created adjacency matrix A^t, the active nodes s^t sampled on the previous step, the original node degrees d^0 and the node-attributes X, a new adjacency matrix A^{t-1} is generated by sampling from the denoising model $\tilde{p}_\theta(A^{t-1}|A^t, s^t, d^0, X)$, which is defined in Eq. 3.4.

Algorithm 3.2: Topology reconstruction using the node-guided diffusion model

Input: Original graph $G(A_{OG}, X)$, diffusion steps T, distributions $q(s^t|d^t, d^0)$, guidance strength w, denoising distribution $\tilde{p}_\theta(A^{t-1}|A^t, s^t, d^0, X)$

Output: Reconstructed adjacency matrix A_{rec}

$d^0 = deg(A_{OG})$

A^T is empty matrix like A_{OG}

for $t = T \dots 1$ **do**

  Draw $s^t \sim q(s^t|deg(A^t), d^0)$

  Draw $A^{t-1} \sim \tilde{p}_\theta(A^{t-1}|A^t, s^t, d^0, X)$

$A_{rec} = A^0$

Return A_{rec}

3.2.2 Monte Carlo Simulation

The idea behind the overarching alliGATOR model, the inference of which is visualised in Fig. 3.1, is that once the edges of the graph are reconstructed using the node-guided diffusion model, they are compared to the original ones. If there is a discrepancy between the original and the reconstructed topology, this is indicative of an anomaly in the graph. The node-guided diffusion model generates a single graph topology with definitive edges. Two *intertwined facts*, however, make a single reconstruction insufficient and motivate a Monte Carlo approach.

1. **Multiple valid topologies**

   For a given set of nodes with node degrees, there can be several ways to correctly connect them while obeying the structural constraints encoded by the node attributes. In Fig. 3.4 is an example of this. Both Case 1 and Case 2 follow the rules that every green node attaches to exactly one green and one purple node, while no two purple nodes may connect. Neither is the "correct" way of connecting the nodes, therefore, even if our model learns to generate graph topologies that are inline with the node attributes constraints, it is possible that an anomaly is found where there is non.

2. **Non-determinism**

   To perform the reconstruction, on each time step the diffusion model uses a link predictor to generate edge probabilities (only between active nodes) and those probabilities are used as a parameter of a Bernoulli distribution from which a single sample is drawn. Even if the model learns to reconstruct the edges correctly, because the process of sampling is stochastic, given the same nodes, the model may generate different topologies.

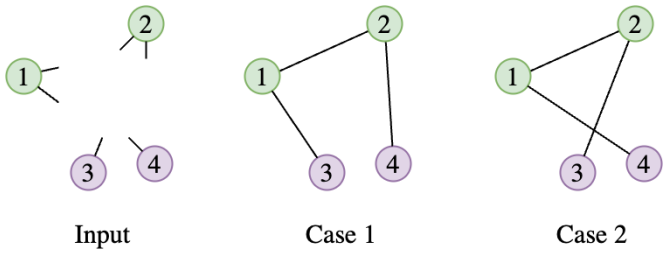


Figure 3.4: Why we need Monte Carlo simulations

Therefore, we need the probabilities of the edges to determine if there is an anomaly, not a single binary realisation. A natural but impractical choice is to analytically calculate them using the probabilities from each denoising step. Unfortunately, this is too complex to do. Although within a single denoising step the edges are independent, new edges (from denoising step $t-1$) depend on existing ones (from denoising step t). This dependence is not directly evident, but it is twofold. Once an edge is added, the node degree d^t is increased, and the node is less likely to be selected as active. More importantly, even when a node is selected as active, the existing edges are used by the graph neural network of the link predictor. We, therefore, adopt a pragmatic Monte-Carlo alternative: run the topology reconstructor MC times, record how often each edge appears, and treat that empirical frequency as the edge probability to be used in the anomaly score calculations.

Accounting for node degrees The Monte Carlo simulations address the problem of multiple valid topologies and non-determinism, however, we need to adjust the edge probabilities with respect to the node degrees. That is necessary because edges between nodes with high node degrees are sampled more often than edges between nodes with low node degrees. Highly connected nodes from the original subgraph are active more often, because nodes are active when there is a difference between the original node degree and the current one. Every time nodes are active, all edges between them are sampled, even if they were previously sampled unsuccessfully. Therefore, let

us assume we have an edge with probability of $p(edge) = 0.1$. If this probability stays at 0.1 across all time steps, the probability of that edge being sampled at least once over T steps is $p = 0.1 + 0.9 * 0.1 + 0.9^2 * 0.1 + \cdots + 0.9^{T-1} * 0.1$. This can be expressed differently as the complement of the probability that an edge is not sampled once over T steps, which is $p = 1 - 0.9^T$. Therefore, as T increases the probability of sampling the edge at least once also increases.

To account for the node degrees, we first calculate the expected number of edges between two nodes. Given a graph $G(V, E)$, let d_i denote the degrees of node i, and $m = |E|$ denote the number of edges in the graph. Then, the expected number of edges between two nodes i and j is $\frac{d_i d_j}{2m-1}$. From node i, trying to connect one of its degrees to another node, the chance of choosing node j is $\frac{d_j}{2m-1}$, as there are $2m$ degree ends in the graph and we are trying to connect one of them. Given that node i has d_i number of degrees to be connected, the expected number of edges between two nodes is $\frac{d_i d_j}{2m-1}$. This formulation is not the edge probability, however, in the limit of large m, the expected number of edges and the probability of an edge become equal. Following the same reasoning, the -1 in the denominator is ignored, therefore, the formulation is $p(e_{i,j}) = \frac{d_i d_j}{2m}$ (Newman, 2018).

Therefore, we can calculate the probability our model gives in addition to the probability of an edge due to node degree, as the difference between the Monte Carlo probability and the probability of the edge due to degrees. Let $W \in \mathbb{R}^{n \times n}$ be the probability matrix we obtained from the Monte Carlo simulations, which is the frequency of occurrence of each edge and n is the number of nodes in the graph. Then the score our model predicts is $L_{i,j} = W_{i,j} - \frac{d_i d_j}{2m}$. When the score $L_{i,j}$ is larger than zero, that means the model is confident the edge (i, j) should exist. While a score $L_{i,j}$ of less than zero indicates the edge (i, j) should not exist.

3.2.3 Anomaly Detection using the alliGATOR

In the previous subsections, we explored how the node-guided diffusion model generates edges, how to obtain the edge probabilities using Monte Carlo simulations, and how to normalise them with respect to the node degrees. With that in mind, this section focuses on the use of the topology reconstructor for anomaly detection. This section presents how the model can be used for node and edge anomaly detection. The difference between the node and edge variants is the preprocessing of the input graph and the different way of calculating the anomaly score given the edge probabilities.

3.2.3.1 Node Anomaly Detection using the alliGATOR

For node anomaly detection, the alliGATOR model works in the following way. To calculate the anomaly score of a central node c from the input graph $G(V, E, X)$ the model first generates its subgraph. The subgraph of a node c is induced by the K most important neighbour nodes of c, K_c, and $\{c\}$. The ranking of the nodes by importance is done by personalised PageRank, discussed in Section 2.1.2. The induced subgraph is denoted by $G_c(V_c, E_c, X_c)$, where $V_c = K_c \cup \{c\}$ is the set of nodes in the subgraph, $E_c = \{\{e_{v_i,v_j}\} \in E | v_i, v_j \in V_c\}$ is the set of edges, and $X_c = \{x_{v_i} \in X | v_i \in V_c\}$ are the node attributes for the nodes in the subgraph. The node-guided diffusion model for topology reconstruction receives as input the nodes with their attributes V_c, X_c, and the extracted node degrees d_c from the set of edges E_c. Note that the denoising model does not receive the original set of edges. The model then reconstructs the subgraph's edges in T diffusion steps and outputs the reconstructed edge set E_{c-rec}. The denoising process is repeated in parallel MC number of times. The edge probabilities are obtained from the frequencies of occurrence in the reconstructed topologies (previously denoted by W), and are further adjusted with respect to the node's degrees. The reconstructed edges with the degree-adjusted scores are denoted by L, where $L_{i,j} = W_{i,j} - \frac{d_i d_j}{2m}$, for $i, j \in N_c$.

A node is considered anomalous if its activity (in this case edges) is out of the ordinary for its type (node attributes). Therefore, to calculate the anomaly of a node c we first calculate the score of its connections (activity), denoted as U_c. The anomaly score is the complement of that score, denoted as $Anomaly_c$. The formula expressing this is presented in Eq. 3.5. The direct neighbours of node c in the original subgraph $G_c(N_c, E_c, X_c)$ are denoted by $\mathcal{N}(c)$. The score of the activity of node c is denoted as U_c. To calculate U_c, we sum the degree-adjusted scores L of the edges between c and the original neighbours of c : $\mathcal{N}(c)$ and divide them by the number of neighbours node c originally had.

$$\begin{aligned} U_c &= \frac{1}{|\mathcal{N}(c)|} \sum_{j \in \mathcal{N}(c)} L_{c,j} \\ Anomaly_c &= 1 - U_c \end{aligned} \tag{3.5}$$

However, because of the node-degree adjustment of the edge probabilities, even when the model predictions align perfectly with the original edges, the score of the node activity can never be one, therefore the anomaly score can never be zero. Therefore, we need to normalise it. To do so, we need to find when the score of the node's activity should be one.

The score of a node's activity should be equal to one when the reconstructed topology always aligns with the original one. Therefore, the Monte Carlo simulations would result in a binary matrix $W_{i,j}$ that has ones for all the edges in E_c. Equation 3.6 presents the maximum achievable score of a node's activity given that the model predictions and the original subgraph align perfectly (denoted by Y_i for node i). In the equation, $\mathcal{N}(i)$ denotes the neighbours from the original subgraph of node i, and d_i the node degree of node i from the original subgraph.

$$
\begin{aligned}
Y_i &= \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} L_{i,j} \\
Y_i &= \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} W_{i,j} - \frac{d_i d_j}{2m} \\
W_{i,j} &= 1 \ \forall j \in \mathcal{N}(i) \\
Y_i &= \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} W_{i,j} - \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} \frac{d_i d_j}{2m} \\
Y_i &= \frac{1}{|\mathcal{N}(i)|} |\mathcal{N}(i)| - \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} \frac{d_i d_j}{2m} \\
Y_i &= 1 - \frac{d_i}{|\mathcal{N}(i)| 2m} \sum_{j \in \mathcal{N}(i)} d_j \\
d_i &= |\mathcal{N}(i)| \\
Y_i &= 1 - \frac{1}{2m} \sum_{j \in \mathcal{N}(i)} d_j
\end{aligned}
\tag{3.6}
$$

Therefore to normalise the score of a node's activity, we must divide it by $1 - \frac{1}{2m} \sum_{j \in \mathcal{N}(i)} d_j$. With the above in mind, the normalised node anomaly score is presented in Equation 3.7, where S_c denotes the score of node c activity, and NA_c denotes the anomaly score of node c.

$$
\begin{aligned}
U_c &= \frac{1}{|\mathcal{N}(c)|} \sum_{j \in \mathcal{N}(c)} L_{c,j} \\
S &= \frac{U_c}{1 - \frac{1}{2m} \sum_{j \in \mathcal{N}(c)} d_j} = \frac{\frac{1}{|\mathcal{N}(c)|} \sum_{j \in \mathcal{N}(c)} L_{c,j}}{1 - \frac{1}{2m} \sum_{j \in \mathcal{N}(c)} d_j} \\
NA_c &= 1 - S
\end{aligned}
\tag{3.7}
$$

The score S measures the "level of agreement" between the original adjacency matrix A and the reconstructed one W. The score S is upper-bounded by 1, but it can be less than 0. Therefore, the node anomaly score is lower-bounded by 0, but can be more than 1.

3.2.3.2 Edge Anomaly Detection using the alliGATOR

The process of detecting edge anomalies is simpler compared to the above-discussed node anomaly detection. For an edge (i, j) the subgraph is induced from the K most important neighbours of the i node and the K most important neighbours of j, denoted by K_i and K_j. We used personalised PageRank to obtain the importance rank for each node. Therefore, let $G_{(i,j)}(V_{(i,j)}, E_{(i,j)}, X_{(i,j)})$ be the subgraph of edge (i, j). Then $V_{(i,j)} = K_i \cup \{i\} \cup K_j \cup \{j\}$ are the set of nodes, $E_{(i,j)} = \{\{e_{y,z}\} \in E | y, z \in N_{(i,j)}\}$ are the set of edges, and $X_{(i,j)} = \{x_{v_y} \in X | v_y \in V_{(i,j)}\}$ are the node attributes. The node-guided topology reconstructor receives as input the nodes $V_{(i,j)}$, their attributes $X_{(i,j)}$, and the node degrees extracted from $E_{(i,j)}$, and reconstructs the subgraph's edges. The process is repeated several times and the probabilities for each edge are computed as the frequency of occurrence over the number of repetitions, denoted as W.

An edge is anomalous if it is between nodes that do not typically interact. And an edge between those nodes should have a low probability. Therefore, we calculate the anomaly score of the central edge (i, j) as the complement of the probability of the edge. The formulation of the edge anomaly is presented in Eq. 3.8. The anomaly score, therefore, lies within the range $[0, 1]$.

$$Anomaly_{(i,j)} = 1 - W_{i,j} \tag{3.8}$$

3.3 Model Implementation

In the alliGATOR model, all learnable parameters are contained within the node-guided topology reconstructor. At the core of this component is a link predictor, implemented using a graph neural network (GNN). The GNN outputs node embeddings used to calculate the edge probabilities between active nodes, denoted as μ_θ in Eq. 3.1. The input of the model to the node-guided network consists of the adjacency matrix A^t, the active nodes s^t, the node degrees of the original subgraph d^0, the node attributes X, and the time step of the diffusion process t.

$$\begin{aligned} p_\theta(A^{t-1}|A^t, s^t, d^0, X) &= \prod_{i,j;i<j} p_\theta(A^{t-1}_{i,j}|A^t_{i,j}, s^t_{i,j}, d^0, X) \\ p_\theta(A^{t-1}_{i,j}|A^t_{i,j}, s^t_{i,j}, d^0, X) &= \mathcal{B}(A^{t-1}_{i,j}|\, s^t_{i,j}\, \mu_\theta(A^t, s^t, d^0, X, t) \;+\; (1 - s^t_{i,j})\, A^t_{i,j}) \\ s^t_{ij} &= s^t_i s^t_j \end{aligned}$$

(3.1 copies from page 29)

We have followed closely the architectural design of the original EDGE model, as presented by Chen et al. (2023). The GNN is a graph attention network which consists of L message passing blocks, also referred to as layers. Each message passing block takes four inputs : (i) the node features $\mathbf{Z}$, (ii) the time embedding t_{emb}, (iii) the global context features $\mathbf{c}$, (iv) the node hidden state features $\mathbf{H}$. The global context features, sometimes called the master node, can act as a bridge between nodes that are not directly connected to pass information about the rest of the graph. They are used to create richer and more complex representations (Gilmer et al., 2017). In our use case, they are useful at the beginning of the generation process when the graph does not have any edges. A sinusoidal position embedding is applied to the time step t. This allows the model to encode time uniquely and to generalise to time steps beyond those seen during training (Vaswani et al., 2017).

The message passing block then updates the node features, the context features and the hidden state features. After all L blocks have iteratively updated the node features, the probabilities ($p_{i,j}$) of the edges between the active nodes ($(i,j) \in \{(i,j)|s_{ij}^t = 1, 1 \leq i < j \leq V\}$) are computed using a multilayer perception.

The initial node features Z^0, passed to the first message passing block, are constructed by concatenating the embeddings of the degree sequences $\mathbf{d}^t$ and $\mathbf{d}^0$, the embedding of the binary vector of active nodes $\mathbf{s}^t$, and the embedding of the node features $\mathbf{X}$. The global context features are initialised by the mean node features Z^0, and the hidden state features are initialised as Z^0. The process is formulated in Eq. 3.9.

$$
\begin{aligned}
\mathbf{Z}^0 &= \text{emb}(\mathbf{d}^t) \parallel \text{emb}(\mathbf{d}^0) \parallel \text{emb}(\mathbf{s}^t) \parallel \text{emb}(\mathbf{X}) \\
t_{\text{emb}} &= \text{emb}(t) \\
\mathbf{c}^0 &= \text{mean}(\mathbf{Z}^0) \\
\mathbf{H}^0 &= \mathbf{Z}^0 \\
\mathbf{Z}^l, \mathbf{c}^l, \mathbf{H}^l &= block_l(\mathbf{Z}^{l-1}, t_{\text{emb}}, \mathbf{c}^{l-1}, \mathbf{H}^{l-1}), \text{ for } l = 1, \ldots, L \\
p_{(i,j)}^{t-1} &= \text{mlp}(\mathbf{Z}_i^L, \mathbf{Z}_j^L) \text{ for } (i,j) \in \{(i,j)|s_{ij}^t = 1, 1 \leq i < j \leq V\}
\end{aligned}
\tag{3.9}
$$

A single message passing block contains a message passing layer, denoted as mpl, and a gated recurrent unit, denoted as gru. A visualisation of the message passing block is presented in Figure 3.5. The message passing layer incorporates attention mechanisms, and the number of heads in each layer are passed as a hyperparameter. The gated recurrent unit, initially introduced by Cho et al. (2014) for sequential data, is a gate-based mechanism to selectively update the hidden state at each time step allowing the model to retain important information while discarding irrelevant details. In the context of graph neural networks, gated recurrent units are employed to mitigate over-smoothing and reduce the impact of noisy neighbourhood information (Huang and Carley, 2019).

$$
\begin{aligned}
\mathbf{Z}^l &= \mathbf{Z}^l \parallel t_{\text{emb}}\mathbf{1}^T \\
\mathbf{Z}^l &= \text{mpl}(\mathbf{Z}^l, \mathbf{A}^t) \\
\mathbf{Z}^l, \mathbf{H}^{l+1} &= \text{gru}(\mathbf{Z}^l, \mathbf{H}^l) \\
\mathbf{c}^{l+1} &= \text{mlp}(\text{mean}(\mathbf{Z}^l \parallel \mathbf{c}^l)) \\
\mathbf{Z}^{l+1} &= \mathbf{Z}^l + \mathbf{c}^{l+1}\mathbf{1}^T
\end{aligned}
$$

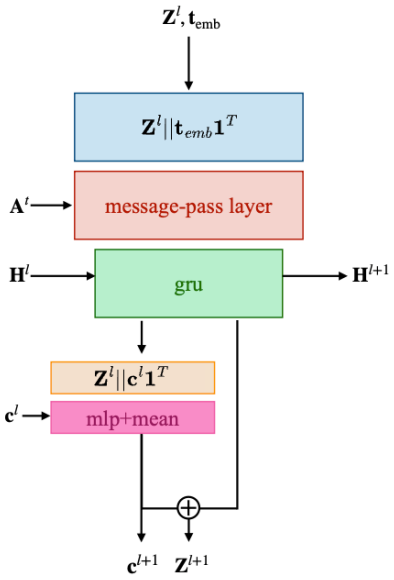


Figure 3.5: Operations in a single message passing block. Adopted from Chen et al. (2023)

During topology reconstruction, on every diffusion step, the computations presented above are repeated twice. Once **with** the node features, and once **without** the node features. When computing the edge probabilities without the node features, the same graph neural network is used with null tokens for node features. The probabilities used for the Bernoulli sampling of each edge are computed as a linear combination between the conditioned and unconditioned GNN result, as described in Eq. 3.4.

3.4 Model Complexity

Model complexity is a key consideration in applications which manage or host large amounts of data, particularly where the deployment environment involves processing data at the scale of millions. In the financial use case, there are millions of accounts (nodes) making tens of millions of transactions a day (edges). The alliGATOR anomaly detection pipeline begins by inducing subgraphs for every instance of interest, which requires the use of personalised PageRank to determine which are the most important nodes. The computational complexity of personalised PageRank is linear in the number of nodes in the input graph – $O(n)$, where n is the number of nodes (Chung, 2014). We require the computation of personalised PageRank for each node, however, they can be treated simultaneously, therefore, given perfect parallelism, the complexity is $O(n)$. After the subgraphs are generated, and stripped of their edges, the topology is reconstructed using the node-guided topology reconstructor. The reconstructor performs three actions at each denoising step: (i) it generates the node embeddings using a graph neural network, (ii) it predicts the active nodes, and (iii) it predicts the edges between the active nodes. The graph neural network used in the model is a graph attention network which has a time complexity of $O(n' \cdot f \cdot f' + m' \cdot f')$ for each layer, where n' is the number of nodes in the instance subgraphs, f is the number of input features, f' is the number of output features, and m' is the number of edges in the instance subgraphs (Brody et al., 2022). Predicting the active nodes requires $O(n')$ operations. Lastly, it takes $O(k^2)$ operations to predict the links between k active nodes. Therefore, the time complexity over T diffusion steps is $O(T \cdot \max(k^2, m))$. The generation is repeated Monte Carlo number of times, however, this generation can be done in parallel. Given perfect parallelism the time complexity of the whole model is $O(T \cdot \max(k^2, m))$.

3.5 Model Hyperparameters

This section presents the hyperparameters of the alliGATOR model and the node-guided diffusion model. Therein we discuss information regarding the general effect each parameter has on the performance of the models, how to choose the hyperparameter values, and how each parameter influences the model complexity.

Parameter K determines the number of nodes used for inducing the subgraphs around each instance of interest (hereby-after referred to as instance subgraphs). The procedure of inducing the instance subgraphs using personalised PageRank is described in detail in Section 3.2.3. The value of K must be large enough that the instance subgraphs have nodes with varying degrees. If the input

graph is sparse, and K is close to the mean number of node degrees, then there is a possibility that the neighbours of the central node do not have edges between them. This will result in a subgraph with a single node connected to many nodes with a single node degree (star graph). Detecting node anomalies in such sparse instance subgraphs is not recommended using the proposed topology reconstructor; as there are not enough nodes with multiple node degrees to allow the model to make connections between. Therefore, the performance of the model would improve in a proportional manner with the value of K. It should also be noted, increasing K also increases the complexity of the model, as there are more nodes in the instance subgraphs to potentially become active and more edges to be used by the graph neural network. Therefore, we want to find a middle ground where the model has enough nodes to make connections, without compromising efficiency.

The number of diffusion steps T determine how many nodes are actively participating in the edge prediction. Contrary to traditional diffusion models for image generation, where more diffusion steps yield better results, the performance of the node-guided topology reconstructor model does not improve with large T. If the instance subgraph is sparse and T is large, then the model identifies only a few active nodes to predict edges between them. This may lead to poor performance. Therefore, setting T is closely related to the size and density of the instance subgraph (i.e. the choice of K). Importantly, T should be less than the highest node degree in the instance subgraph. The number of diffusion steps influences the model complexity in two ways – (i) fewer diffusion steps decrease complexity linearly (as the number of iterations decreases), however, (ii) as the diffusion steps decrease, the number of active nodes per time step increases, and therefore, the complexity grows.

The noise schedule β_t governs the rate of diffusion and denoising the graph. During the reconstruction, it governs the number of active nodes (Eq. 2.20 and 2.21). In generative diffusion research, the most prominent noise schedulers are either a linear noise scheduler or a cosine scheduler. Linear schedule increases the noise uniformly, while a cosine one adjusts the noise increment based on a cosine function, leading to slower changes at the start and end of the process. In the context of the graphs, we seek a uniform denoising, otherwise, we may face the issue of too few active nodes. The linear noise schedule is also preferred from a computational complexity perspective, because the cosine schedule leads to steeper change. This means more active nodes in a single diffusion step in the middle of the diffusion process.

From an architectural perspective the **hyperparameters of the graph neural network** discussed in Section 3.3 include: embedding dimension, number of layers (also called message passing

blocks in Section 3.3), number of attention heads in each layer, dropout rate, activation function, and time embedding. These parameters should be set in accordance with the type and size of input data. For the training of the model, the parameters are batch size, learning rate, optimiser, and patience during training. Additionally, parameter p_{uncond} is the percentage of the training graphs whose node attributes are replaced with a null token. It is recommended to be set to 2% (Ho and Salimans, 2021) (also discussed in Section 2.3.1)

Lastly, once the topology reconstructor is trained, there are two parameters to be adjusted for the anomaly detection, **the guidance strength** w (Eq. 3.4) and **number of Monte Carlo simulations**. When the guidance strength w is set to 0, the model does not include the probabilities of the unconditioned link predictor when sampling the edges. However, as w increases, the model actively reduces the probability of edges proposed by the model that ignores the node features. Regarding the number of Monte Carlo simulations, while generally more simulations lead to better approximations; perfect parallelism is rarely achievable in practice, and the model's complexity increases linearly with the number of simulations. Therefore, the goal is to find a balance between a good approximation of the edge probabilities and maintaining manageable computational complexity.

CHAPTER

FOUR

EVALUATION

In this chapter, we aim to answer the two sub-questions proposed in the Introduction. Namely, SQ1: Can node anomaly be detected by comparing the original induced subgraph with a reconstructed one, that is generated by a diffusion model guided by the original nodes' attributes and degrees? SQ2: Can edge anomaly be detected by comparing the original induced subgraph with a reconstructed one, that is generated by a diffusion model guided by the original nodes' attributes and degrees?

This is done via the execution of two comparative experiments between the alliGATOR model and the Isolation Forest model. To do this, we must discuss the crafted synthetic data generation method, including our reasoning for generating a synthetic dataset (Section 4.1). Then, the evaluation metrics are presented in Section 4.2. The parameters used for Isolation Forest and the alliGATOR model are discussed in Section 4.3. The Chapter continues with an analysis of the training of the node-guided topology reconstruction (Section 4.4). The results of the experiments and the model analysis are presented in Sections 4.5 and 4.6. The Chapter concludes with the discussion in Section 4.7.

4.1 Dataset

Although the alliGATOR model is an unsupervised model, its evaluation requires a labelled graph dataset to provide ground-truth labels for the statistical analysis. We initially explored publicly available datasets, but found none that met our requirements – the presence of meaningful node features and clearly labelled anomalous nodes or edges. For instance, the Elliptic++ dataset (Elmougy and Liu, 2023) has sufficient node features, but lacks explicit labels for all instances and is tailored

to a very specific domain (money laundering). Another example is DGraph dataset (Huang et al., 2022b) which has partially labelled nodes and claims that the anomalous nodes have a different structure than normal ones, however, the majority of the node features are *partial*, with many missing values. While our model could *theoretically* be applied to these datasets, they are not suitable for rigorous evaluation. Additionally, we found no datasets with both node and edge anomalies, therefore, if we were to use publicly available anomalous datasets we would have used significantly different ones, thereby, making the comparison between the two variants of the model difficult.

Given the limitations, we opted to generate a synthetic dataset, allowing us to control key aspects of the data, such as node features, their types and frequencies, intra- and inter-node relations, type-specific degree distributions, and the proportion of anomalous instances. Injecting the anomalies ourselves meant that we have reliable ground-truth labels for both node and edge anomaly detection. By generating the data ourselves, we could systematically evaluate and compare the performance of both node and edge anomaly detection variants of the model.

We propose a model for network generation in which nodes are assigned to multiple classes, and the connections between nodes are governed by the relations between these classes. Although the alliGATOR model is designed to handle arbitrary node attributes – regardless of type or dimensionality – we restrict the synthetic graph generation process to graphs in which each node has a single categorical attribute. Therefore, the first parameter of the network generation model is the number of node classes, denoted by C. For each ordered pair of classes, there is a parameter that defines how nodes from one class interact with nodes from the other, resulting in C^2 interaction parameters. These interactions are directed and can follow one of three generative mechanisms: the Barabási–Albert model (Barabási and Albert, 1999), the configuration model (Newman, 2018), or the Erdős–Rényi random graph model (Erdős and Rényi, 1961), previously discussed in Section 2.1. Consequently, the degree distribution of the source nodes depends on the chosen model and may follow a power-law, uniform, or normal distribution, respectively.

The interaction parameters can be "None", indicating that no directed interaction between nodes from the given classes. Otherwise, the interaction parameter must include the specific parameters required by the selected generative model. For example, if the relation between node classes X and Y, follows the Erdős–Rényi model for bipartite graphs with a parameter 0.01, then the distribution of the node degrees would be normal [1].

[1]The distribution is actually Binomial, but when the number of trials in a binomial experiment is large enough, the binomial distribution can be approximated by a normal distribution

After generating the individual subgraphs corresponding to each class interaction, these subgraphs are merged (based on node id) to form a single graph containing all nodes and relations. We then remove edge directions, as the current implementation of the alliGATOR model requires the input graph to be undirected.

Injecting anomalies is done once the graph with all the nodes and the desired relations is generated. We create separate graphs for the node anomaly and the edge anomaly experiment. These experiments will be used to answer sub-question one, and sub-question two, respectively. For the node anomaly experiment, we inject node anomalies in the graph by changing the node type (i.e. category) of a percentage of the nodes. However, a node is anomalous only when its pattern of activity (i.e. its connections) does not match its type. Therefore, when changing the node categories, the newly assigned category must have a different pattern to the original one. For example, if nodes from type X connect to both node types Z and Y, but node types Z and Y connect only with nodes from type X (and do not connect with their own node type), then changing the node category from Z to Y will not make the node anomalous. For the edge anomaly experiment, we add anomalies by adding edges between nodes that do not have a relation in the original graph. Using the earlier example that would be an edge between nodes Z and Y. To ensure proper distribution of anomalous instances throughout the training and evaluation of the model, we must establish a data split, and only after add the anomalous instances.

Splitting the data is performed differently, depending on whether the setting is transductive or inductive. In the context of graphs, transductive approaches require that the entire graph is observable during training, validation, and testing. As an example, if the task is node classification under a transductive setup, then the node labels are partitioned into training, validation, and test sets. During training, node embeddings are computed using the full graph structure (i.e., all nodes and edges), but the model is optimised only on the node labels that were in the training split. Similarly, at evaluation and testing, embeddings are still computed using the entire graph, and the model is evaluated using the labels from the validation and test splits, respectively. Transductive approaches outperform inductive ones with regard to prediction accuracy, however, they do not generalise to unseen graphs, and typically incur higher training costs. Given that our aim is to build a generalisable anomaly detection model, which does not require retraining when new nodes are added, we adopt an inductive approach for the link prediction.

Inductive approaches learn to predict labels, or in our case edges, for unseen data instances. In the context of graphs, inductive models are trained on multiple graphs. Therefore, to simulate an inductive learning scenario using a single graph, we partition the edges to create disjointed subgraphs for training, validation, and testing. Generally, the preferred approach of splitting graph data is temporally, by ordering the edges chronologically and taking the edges that appeared in the last X% of the time steps. However, as we do not model the graph in time, we sample the edges randomly. The random selection of edges is justified, given the constant set of nodes in financial use-cases; where nodes represent accounts, which are static throughout all time steps. As in real life, sampling edges at random is akin to taking a part of a temporal split, where these random edges correspond with the latter X% of a temporal split. Lastly, to ensure no information leakage from the training to the evaluation, the nodes used for evaluation are removed from the training graph. In our case this is not required, as we are working with categorical nodes, that do not have uniquely identifying features.

The instantiation of the synthetic data generation model we have used is presented in Table 4.1. We generate a graph with 1,000 nodes, with three possible categories. For ease of discussion and visualisation, we assign the following colours to the categories: “blue”, “orange”, and “grey”. The distribution of the nodes over the categories is 25%/35%/40%, respectively. Given the three node types, there are nine possible directed relations between them ($blue \rightarrow blue$, $blue \rightarrow orange$, $blue \rightarrow gray$, $orange \rightarrow blue$, etc). As previously introduced, to simulate different real-world scenarios, each relation is generated independently. These individual graphs are then merged (based on node ID) to form a single graph containing all generated relations. Each relation is generated using one of three algorithms for graph generation, allowing for different degree distributions of the source (left-hand side in the table) node type. The possible degree distributions are power-law, uniform, or normal, generated respectively by the Barabási–Albert model(Barabási and Albert, 1999), the configuration model (Newman, 2018), or the Erdős–Rényi random graph model(Erdős and Rényi, 1961).

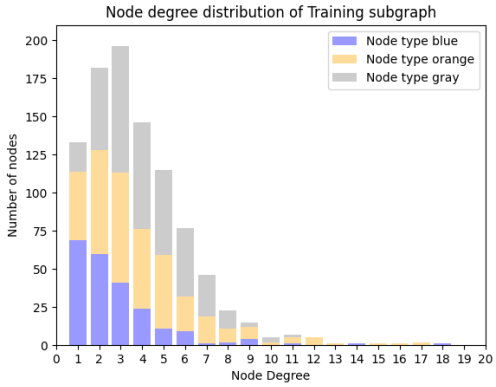
	Blue	Orange	Grey
Blue	Barabási–Albert, 3	None	Erdős–Rényi, 0.01
Orange	None	None	Configuration Model, 7
Grey	None	Barabási–Albert, 8	None

Table 4.1: The algorithms and algorithm parameters used to generate the relations between the different types of nodes

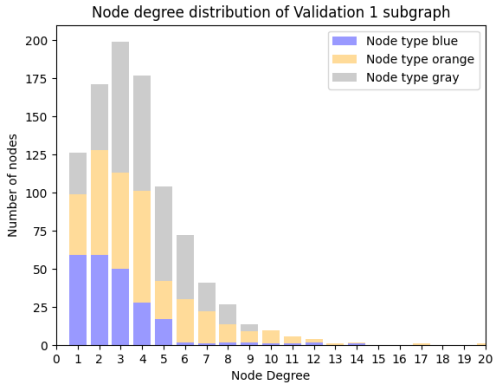
The relation *blue* → *blue* is generated using Barabási–Albert model, adding blue nodes one by one to the relation graph and connecting each new blue node to three already existing ones. The same mode is used for the *grey* → *orange* relation, in this case, we first generate all orange nodes, then we connect each grey node to eight orange nodes, choosing them preferentially based on their node degrees. For the *blue* → *gray* relation we've used Erdős–Rényi with a probability of $p = 0.01$ that an edge between every combination of blue and grey nodes appears. Lastly, the *orange* → *grey* relation is generated using a configuration model, where each orange node has seven free connections that need to be connected to a grey node at random. Although the described relations are directed, after the graphs are merged, the directions are removed, resulting in only three out of six possible undirected relations (*blue* ⟷ *blue*, *blue* ⟷ *grey*, *grey* ⟷ *orange*). **We chose these parameters, because we wanted to create a dense graph, which could withstand the edge splitting required for the training and validation.** The node attributes are the one-hot embedding of the category.

For the evaluation of the alliGATOR model, we employ a four-way edge split comprising 25% for training, 25% each for two validation sets, and 25% for testing. The first validation set is used for early stopping of the training, while the second is used for hyperparameter tuning. Although our choice of equal split is unconventional, using a typical 70%/15%/15% split would result in a dense training subgraph and extremely sparse validation and test subgraphs. The inconsistency could impair generalisation, hinder the early stopping (validation set 1), mislead hyperparameter tuning (validation set 2), and result in unreliable model evaluation (training set). Ideally, edges should be divided into ten sets, with 8-fold cross-validation applied: seven folds for training, one for early stopping, one for alliGATOR hyperparameter tuning and the last for testing. This approach works well with dense graphs, however, given that we must induce subgraphs for each instance of interest, as described in Section 3.2.3, the equal split results in better subgraphs. To verify that the edge

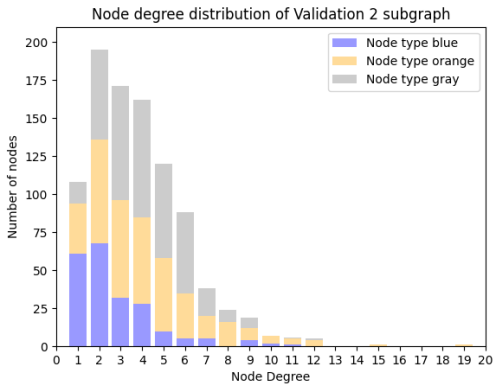
split of the synthetic graph resulted in subgraphs with similar node degree distribution, we plotted the node degree distribution for each of the four subgraphs – training, validation 1, validation 2, and testing. The plots are presented in Figure 4.1, in which the colours correspond to the node types. What we can see is that overall the split resulted in similar node degree distributions for the training and the evaluation subgraphs.



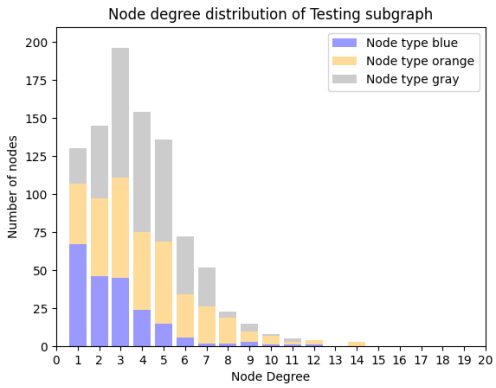
(a) Training subgraph



(b) Validation 1 subgraph



(c) Validation 2 subgraph



(d) Testing subgraph

Figure 4.1: Node degree distribution of synthetic dataset by splits

Once the graphs are split, we add the anomalies. For node anomalies, we changed the colour of 4% of the nodes (~ 40 nodes in each graph), 80% of those nodes are changed from grey to orange and the other 20% of the nodes we changed from orange to grey. We chose those node types, because orange nodes are not connected to blue nodes, whereas grey nodes are, therefore, creating an anomaly. Similarly, grey nodes do not connect with other grey nodes, therefore changing to an orange node, which allows connections to grey nodes, introduces an anomaly. We change colours ensuring double change for a single node is impossible. For edge anomalies, we have added 4% of existing edges as anomalous, split equally between the other three relations, namely $blue \longleftrightarrow orange$, $orange \longleftrightarrow orange$, and $grey \longleftrightarrow grey$.

4.2 Evaluation Metrics

To evaluate the performance of the proposed alliGATOR model, we opted to use the area under the precision-recall curve (AUPRC). This metric is particularly well-suited for the anomaly detection task, because we have a binary classifier with heavily unbalanced data. As discussed in Sections 4.1 only 4% of the total instances represent positive (i.e., anomalous) samples. Traditional metrics such as accuracy or the area under the ROC curve can be misleading, because even a model biased toward predicting the majority class could still achieve deceptively high accuracy. In contrast, the precision-recall curve emphasises the model's ability to correctly identify the minority class. High precision indicates a low false positive rate, while high recall indicates how well the model identifies all relevant instances of the minority class. Therefore, the AUPRC provides a concise and informative summary of this trade-off, making it a more reliable metric for our evaluation. We compute the AUPRC using average precision, rather than interpolation.

4.3 Compared Models

The area under the precision-recall curve, although informative, provides an absolute measure of performance. To contextualise the performance of the alliGATOR model, we compare it against the well-known Isolation Forest (Liu et al., 2008), discussed in Section 2.5. We chose the Isolation Forest algorithm for a few reasons – it is an unsupervised method, it scales well, it has been studied in depth, and it has a well-maintained implementation in the *scikit-learn* Python library. The models are compared using two experiments – node anomaly detection and edge anomaly detection – seeking to answer sub-questions one and two, respectively.

4.3.1 Isolation Forest

Applying Isolation Forest for node or edge anomaly detection, requires a feature vector for each instance of interest to be devised. This is because working solely with the attributes of the nodes/edges disregards the role the instance of interest plays in its local network. There are different ways of generating useful representations, as discussed in Section 2.2. Some notable examples include, Node2vec (Grover and Leskovec, 2016b) and struc2vec (Ribeiro et al., 2017) that create node embeddings based on random walks. Node2vec creates embeddings based on node neighbourhoods, resulting in neighbouring nodes having similar embeddings, whereas struc2vec creates embeddings based on the nodes' structural role. However, neither of those methods differentiates between node types, therefore, they cannot take into account the node features.

The alternative, which is what we used for this thesis, is to generate feature vectors in a similar manner to graph neural networks. For each node, we concatenate its attribute with the aggregated attributes of its neighbours. We experimented with four functions for the AGGREGATION– **sum**, **mean**, **min**, and **max**. The operation is presented in Eq. 4.1, where x_i is the original attribute of node i, and $\mathcal{N}(i)$ is the set of neighbours of node i.

$$x'_i = x_i \,||\, \text{AGGREGATION}_{j \in \mathcal{N}(i)}(x_j) \tag{4.1}$$

For node anomaly detection, the input to the Isolation Forest algorithm are the above devised features x'_i for each node. For edge anomaly detection, we concatenate the devised feature vector of each node of the edge.

The alliGATOR model operates under an inductive setting, thus, it would only be fair to compare it to Isolation Forest trained and evaluated under the same setting. Therefore, after the input graph is split as described in Section 4.1, the node embeddings are generated for each graph separately. The model is trained using the node embeddings from the training dataset, and tested on the nodes from the testing graph. We follow the recommendations of Liu et al. (2008) for parameter settings, using a subsampling size of 256 and 100 trees in the ensemble.

Based on the precision-recall curve and the mean average precision, we found that the best aggregation function is **mean**, closely followed by **sum**. As a result, we use the **mean** operation. Given that in the synthetic dataset the node attributes are one-hot encoded, the node feature vector consists of the attributes of the node and the frequency of each node type in its immediate connections.

4.3.2 alliGATOR Parameter Values

This section presents the hyperparameter values used for the alliGATOR model and our motivation behind setting them at those values. The hyperparameters vary depending on the task – whether it is node or edge anomaly detection. All hyperparameters are listed in Table 4.2.

Parameter K determines the number of nodes used for inducing the subgraphs around each instance of interest. For the synthetic data generation we found that for the node anomaly subgraphs, K of 15 results in diverse instance subgraphs (with respect to node degree). Examples of the subgraphs are presented in Figure 4.2, the first row presents subgraphs around a normal central node, whereas the second one presents subgraphs around an anomalous one. For the edge anomaly detection, as the edge instance subgraphs are induced by the neighbours of both nodes on each side of the edge, we chose a K of 7.

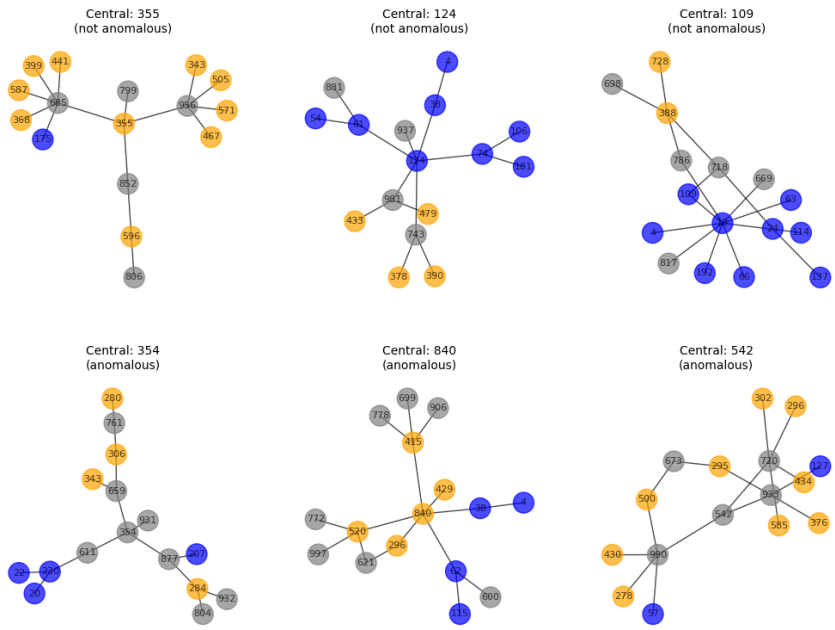
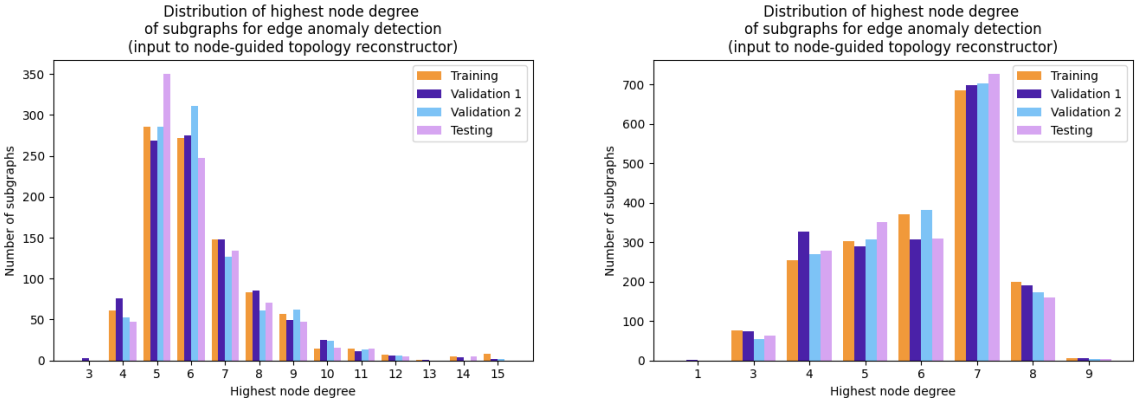


Figure 4.2: Example of subgraphs for node anomaly detection from the synthetic dataset (input to the node-guided topology reconstructor)

Setting the number of **diffusion steps** depends on the highest node degree in the subgraph; however, subgraphs have different degree distributions, but the diffusion step cannot be different for individual instance subgraphs. Therefore, to choose an appropriate value for the hyperparameter,

we plot the distribution of the highest node degree for all instance subgraphs in each split. The plots are presented in Figure 4.3, where the left plot has the values for the node subgraph, and the right plot has the values for the edge subgraphs. Based on the results, we set the parameter to 4 for the node anomaly detection and 5 for the edge anomaly detection. We set the noise schedule to linear for reasons discussed in Section 3.5



(a) Node anomaly detection subgraphs

(b) Edge anomaly detection subgraphs

Figure 4.3: Plots of the maximum node degree of subgraphs for node anomaly detection (left) and edge anomaly detection (right) from the synthetic dataset

Regarding the **architecture of the graph neural network** used for link prediction, we experiment with 3 different settings for the number of message passing blocks and attention heads. For example, the parameter $[3, 1]$ refers to two blocks (layers) with three attention heads in the first layer and one in the second. The rest of the parameters are set to standard values, with embedding dimension of 16, dropout rate of 0.1, SiLU activation function (Elfwing et al., 2018), and sinusoidal positional embedding (Devlin et al., 2019).

For the **optimisation** we use a batch size of 32, learning rate of 10^{-3}, Adam optimiser (Kingma and Ba, 2015), and we train the model with a patience of 30 checked every 3 epochs (or until a maximum of 1000 epochs). p_{uncond} is the percentage of the training graphs whose node attributes are replaced with a null token. We set it to the recommended 2% (Ho and Salimans, 2021) (also discussed in Section 2.3.1).

Lastly, we experiment with three values for the **guidance hyperparameter** w $(0, 0.5,$ and $4.5)$. All values ensure that the diffusion model incorporates the node attributes, which are essential to

our approach. The goal of experimenting with different values is to assess the impact of subtracting the unconditioned model output and to understand how much this subtraction contributes to the overall performance. For the **Monte Carlo simulation** we experiment with $10, 50$, and 100.

	Node Anomaly	**Edge Anomaly**
Instance subgraph		
K	15	7
Diffusion		
Diffusion steps T	4	5
Noise schedule	*linear*	
Architecture		
Embedding dimension	16	
Message passing blocks & attention heads	$[[1], [3, 1], [3, 3, 1]]$	
Dropout rate	0.1	
Activation function	SiLU	
Time embedding	Sinusoidal positional embedding	
Optimisation		
Batch size	32	
Learning rate	10^{-3}	
Optimiser	Adam	
Patience	30	
p_{uncon}	2%	
Anomaly detection		
Guidance w	[0, 0.5, 4.5]	
Monte Carlo simulations	[10, 50, 100]	

Table 4.2: alliGATOR hyperparameters

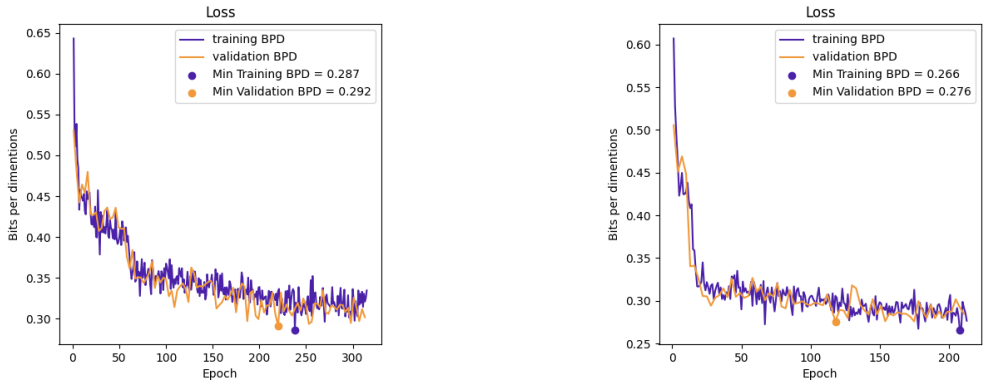
Based on the parameter settings described above, there are 27 possible hyperparameter combinations for the node and edge anomaly. The model is trained for up to 1000 epochs and evaluated every 3 epochs using validation set 1. Early stopping is applied with a patience of 30, meaning

training stops if the validation loss does not improve after 30 validation steps. Once training is completed, validation set 2 is used to compare the different hyperparameter settings. Finally, the test set is used to report performance. Model weights are restored from the checkpoint with the lowest loss on validation set 1.

The optimal configuration for node anomaly detection consists of two message passing blocks with attention heads $[3, 1]$, a guidance strength of $w = 0.5$, and 100 Monte Carlo simulations. For the edge anomaly detection task, the optimal parameters are three message passing blocks with $[3, 3, 1]$ attention heads, guidance strength of 4.5, and 100 Monte Carlo simulations.

4.4 Model Training

To assess how well the alliGATOR model learns, we plot its training and validation loss for both tasks in Figure 4.4. Each plot corresponds to the model trained with the optimal hyperparameters. The x-axis shows the number of epochs, and the y-axis represents the loss in bits per dimension, which is another way of representing the loss taking into account the number of edges over which the loss is calculated. Both plots exhibit a clear decrease in the loss for both the training and the validation indicating effective learning without signs of underfitting or overfitting. In some areas the validation loss is slightly lower than the training one, however, this is likely due to the dropout layer, which is active during training but not during evaluation. This typically leads to marginally lower loss values on the validation set due to the absence of stochastic regularisation.



(a) Node anomaly detection

(b) Edge anomaly detection

Figure 4.4: Training loss of the alliGATOR model

4.5 Results

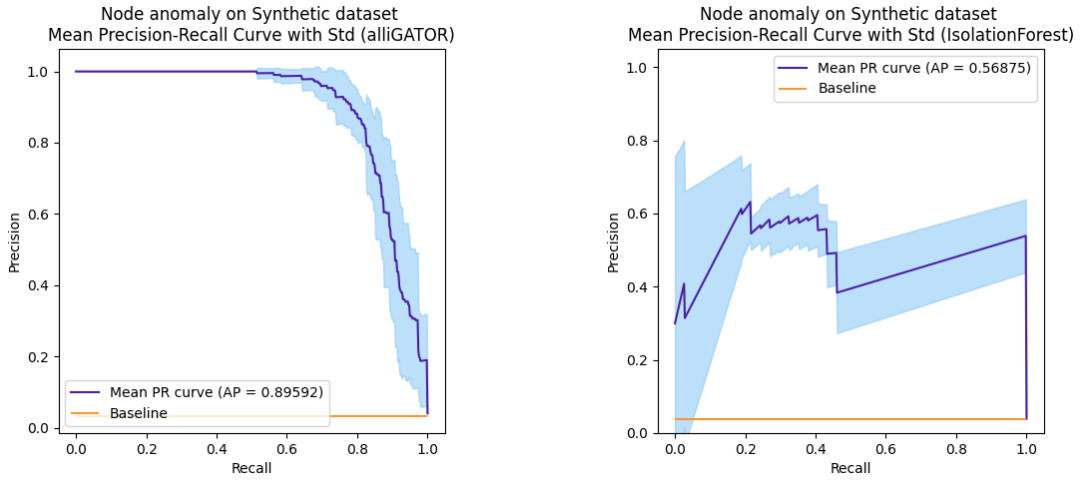
This section details the results of the node anomaly detection experiment and the results of the edge anomaly detection experiment, for each model.

4.5.1 Node anomaly detection

The results of the node anomaly experiment are presented in Figures 4.5 and 4.6. The precision-recall curves of the alliGATOR model and the Isolation Forest are presented in Figures 4.5a and 4.5b, respectively. The plots are generated over ten runs, to account for the non-determinism of the alliGATOR model and the stochastic sampling of instances during training of the Isolation Forest. The figures present the averaged precision-recall curve as the purple line, the standard deviation across the ten runs as a shaded blue area, and the random chance baseline as an orange line. As seen in the Figure 4.5a, the mean average precision of the alliGATOR model is **0.895**, which indicates that the model performs well at distinguishing anomalous instances from normal ones. Figure 4.5b, presents the results of the Isolation Forest, which has a notably lower mean average precision of **0.568**.

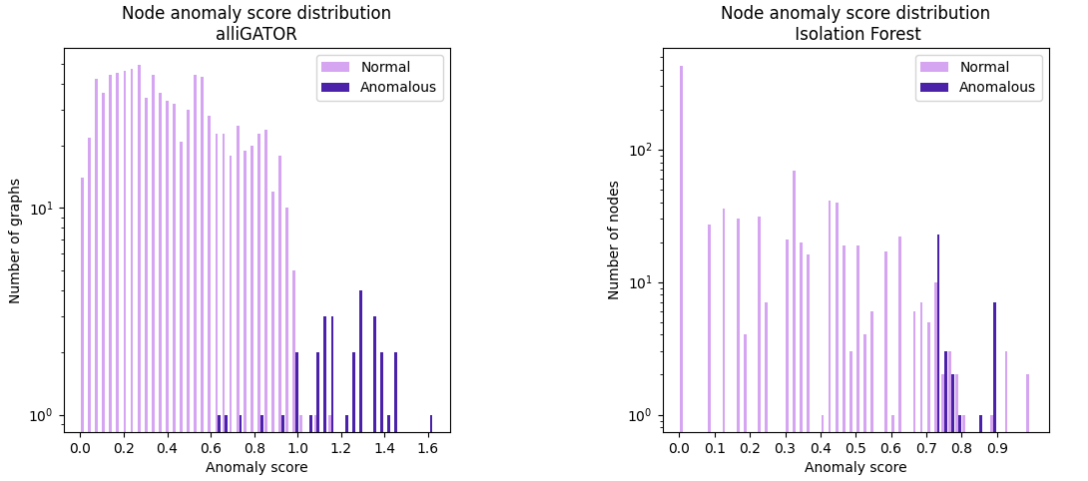
The alliGATOR curve begins with a flat segment at high precision and low recall indicating, that the model correctly identifies the positive instances it predicts. The gradual decrease indicates that the anomaly score distributions of the normal nodes and the anomalous ones do not overlap significantly. However, the low precision for recall near 1 shows that to get all the anomalous nodes correctly classified, the model must incorrectly flag a significant portion of the normal nodes. To support this claim, we show the distribution of the anomaly scores for the normal and anomalous instances of the last run in Figure 4.6a. Overall, the alliGATOR model achieves high and stable anomaly detection performance with low variability.

In contrast, the Isolation Forest precision-recall curve presents erratic behaviour with sharp jumps and dips, indicating poorer performance. The low overall precision indicates that the anomaly score distributions of the normal and anomalous nodes overlap. However, compared to the alliGATOR model, the Isolation Forest has higher precision at recall near 1 (~ 0.55 vs ~ 0.1), indicating that to correctly classify all the anomalous nodes, it does not have to misclassify as many normal nodes as the alliGATOR model.



(a) alliGATOR model (b) Isolation Forest

Figure 4.5: Node anomaly results – Precision-recall curves



(a) alliGATOR model (b) Isolation Forest

Figure 4.6: Node anomaly results – Anomaly score distribution

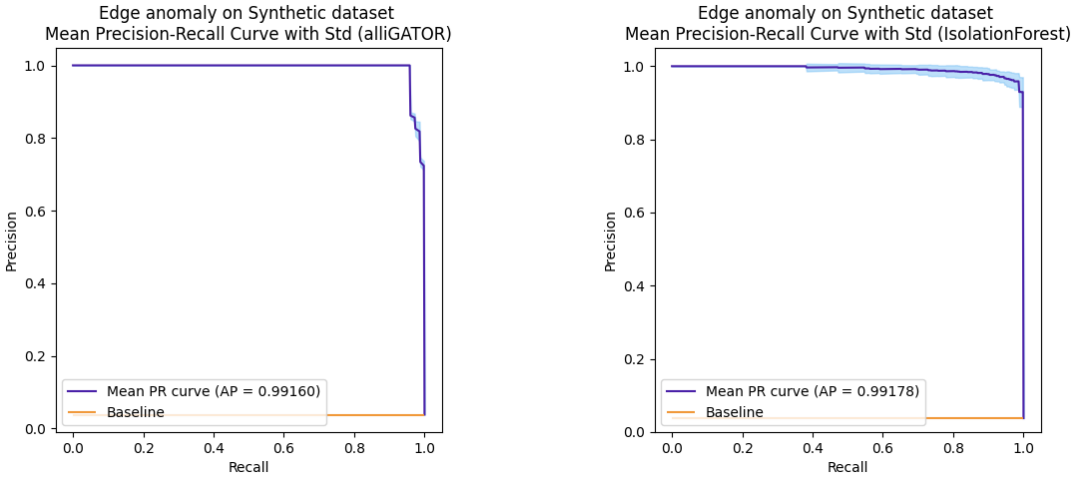
4.5.2 Edge anomaly detection

Figures 4.7 and 4.8 present the results of the edge anomaly experiment. The plots in Figure 4.7 are generated over ten runs, with purple line presenting the averaged precision-recall curve, shaded blue area presenting the standard deviation, and orange line indicating the random chance. Both

models exhibit near-perfect performance, with a mean precision of **0.9916** for the alliGATOR model and **0.9917** for the Isolation Forest. Based on the lower variability, the alliGATOR model presents to be more stable. Although numerically the models have a difference of **0.0001** in favour of the Isolation Forest, their precision-recall curves reveal opposite behaviour (the shapes of the curves are *mirrored*). The alliGATOR maintains a near-perfect precision for all values of recall until ~ 0.9, when it notably drops to a precision of ~ 0.7 for recall of 1. In contrast, the Isolation Forest begins a gradual decrease of precision from recall value ~ 0.4, however, even at recall 1 the precision maintains a high value – around 0.9.

The difference in the curves indicates a difference in the anomaly score distribution between normal and anomalous nodes. The near-perfect precision with a steep drop in the curve of the alliGATOR model indicates the distributions have a minimal overlap with few individual anomalous instances being out of distribution, therefore, classifying them correctly sacrifices the precision. This is evident from the plots in Figure 4.8, showing the anomaly score distribution from the last run. The gradual decrease of the Isolation Forest points to a small overlap of anomaly score distributions, but without any anomalous instances out of distribution.

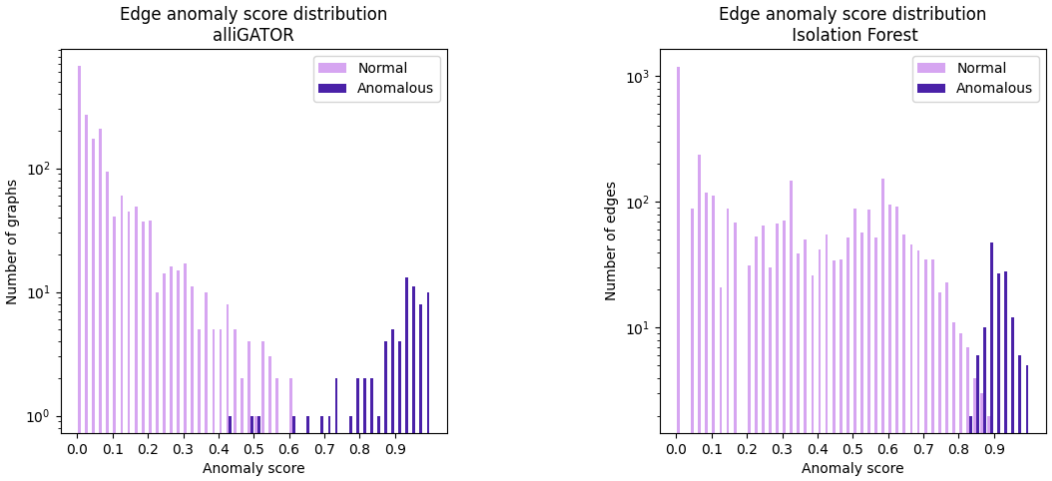
The alliGATOR model is better for applications where it is crucial to avoid false positives, thus, high precision is sought after. The Isolation Forest is better for tasks focused on recall, as the task does not lower the precision too much to achieve a recall of one.



(a) alliGATOR model

(b) Isolation Forest

Figure 4.7: Edge anomaly results – Precision-recall curves



(a) alliGATOR model

(b) Isolation Forest

Figure 4.8: Edge anomaly results – Anomaly score distribution

4.6 Model Analysis

Anomaly score analysis We investigate the impact of the edge probability normalisation on the performance of the alliGATOR model. This normalisation process, described in Sections 3.2.2, is applied during node anomaly detection to adjust for high node degrees. To assess its contribution, we remove the normalisation step and instead use the raw probabilities derived directly from the Monte Carlo simulations when computing anomaly scores. The results of the experiment are presented in Figure 4.9. We compared them to the earlier findings in Section 4.5.1. Without normalisation the model achieves a mean average precision of **0.462**, which is considerably lower than the **0.895** achieved *with* the probability normalisation. Numerically, the result of the *unnormalised* alliGATOR version is lower than the Isolation Forest model (0.568), however, the curve is more stable, with no jumps and lower variability. The notable increase of the model performance with the addition of the edge normalisation is evidence to its importance.

Hyperparameter sensitivity analysis To evaluate the robustness of the alliGATOR model, we investigate the effect of individual hyperparameters on its performance. Each parameter was varied independently while keeping the others fixed. The results are presented in Table 4.3. Increasing the depth of the graph neural network from one to two layers improves the model performance by

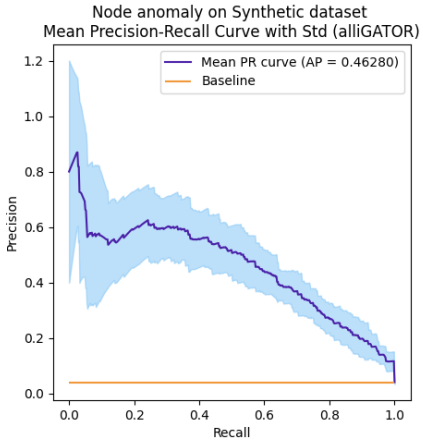


Figure 4.9: Node anomaly detection using alliGATOR without edge probability normalisation

approximately 10%. In contrast, increasing the guidance strength yields minimal improvement and exhibits high variability. Increasing the Monte Carlo simulations from 10 to 50 notably improves the model performance, however, after 50 there are diminishing returns. Overall, the model demonstrates robust performance, as even with a suboptimal hyperparameter configuration, the alliGATOR model outperforms the Isolation Forest baseline on the node anomaly detection task, and maintains high average precision for the edge anomaly task.

Hyperparameter	**Change**	**Node anomaly** Mean improvement (with std. dev.)	**Edge anomaly** Mean improvement (with std. dev.)
Message passing blocks & attention heads	[1] → [3, 1]	+0.136 (±0.035)	+0.106 (± 0.051)
	[3, 1] → [3, 3, 1]	-0.059 (±0.041)	+0.017 (±0.032)
Guidance w	0 → 0.5	+0.02 (±0.043)	+0.092 (±0.031)
	0.5 → 4.5	-0.027 (±0.05)	+0.087 (±0.046)
Monte Carlo simulations	10 → 50	+0.115 (±0.048)	+0.111 (±0.042)
	50 → 100	+0.004 (±0.014)	+0.009 (±0.008)

Table 4.3: Effect of varying individual hyperparameters on model performance. All values indicate the mean average precision improvement ± standard deviation, keeping other parameters fixed.

4.7 Discussion

From the results presented in the previous section, we can deduce that the alliGATOR model shows promising performance. However, the high performance of both models indicates the simplicity of the problems in the synthetic dataset. This is evident from the almost textbook-perfect performance of both models on the edge anomaly task. When compared to the node anomaly, the edge anomaly detection task is considerably simpler. Given the six possible undirected relations between the three types of nodes, three of them are anomalous. Even a model that uses only the node attributes of the edge nodes could be able to identify an anomalous instance with an accuracy of 100%. The node anomaly task is more realistic, with varying node degrees and distributions of neighbouring node types. Below we note some of the important considerations to take into account when experimenting with more complex problems.

The hyperparameter sensitivity analysis demonstrates that the **guidance parameter** w has no significant impact on the performance of the model on the node anomaly detection task, and only a minimal impact on the edge anomaly task. Classifier-free guidance, controlled by parameter w, is generally used to ensure that diffusion models do not disregard conditioning information (Section 2.3.1). In the alliGATOR model, the conditioning information is the node features. The node features are central to the performance of the model because local properties cannot be captured solely by the node degrees. An exception arises in extremely sparse graphs with heavy unbalance in the node degrees. In such cases the model might be inclined to favour the node degrees instead of the node features. As an example, if the instance subgraphs are predominantly consisting of a single node connected to many nodes with a node degree of one, then the model might rely on the degrees more than the node features to predict the connections. Therefore, we believe that in diverse instance subgraphs, where the node features relay more information than the node degrees, the diffusion model *should* naturally attend to the node features without explicit enforcement through classifier-free guidance. This calls for further investigation. Future work in this direction should also account for the latest research in generative diffusion; suggesting the use of a scheduler for the guidance (Malarz et al., 2025).

The number of **Monte Carlo simulations** required for strong model performance is closely related to the edge probabilities produced by the link predictor. In our synthetic data, due to the simplicity of the problem, the model quickly becomes confident and the edge probabilities can be approximated with fewer simulations. However, when the node features are richer, and the local properties of the graph are more complex, we expect the model to assign lower probabilities

with greater variance. Consequently, more Monte Carlo simulations would be required for the accurate approximation of the edge probabilities. We believe this would be prominent in larger, highly diverse instance subgraphs. However, increasing the Monte Carlo simulation increases the complexity linearly. Therefore, a solution might consider a rescaling of the edges using a function of the size of the graph and the complexity of the node features.

The main **limitation of our experimental setup** is that we test the model on a single type of node anomaly. The anomalies we inject are all relation based – nodes with specific node attributes should not interact, and when they do we deem them anomalous. Another type of node anomaly is a high-degree burst, which is an unexpected high node degree in a short time window. The ability of our model to detect such anomalies is dependent on whether the node is anomalous due to its degree or due to the connection made due to the burst. If the connections made during the burst are predominantly between nodes, that do not typically interact, then we hypothesise that our model should detect the anomaly. At first glance, it might seem that because the alliGATOR model takes the node degrees as an input to the link predictor, it strictly adheres to making as many connections as the node degrees. However, preliminary experiments show that the degrees do not force the model into making connections between unsuitable nodes. This is the same reasoning behind the guidance argument made above – if the training instance subgraphs are diverse, then the model lessens its dependence on the node degrees in favour of the node features. Applying the topology reconstructor on a subgraph of an anomalous node with a high degree would result in an emptier reconstructed graph, with fewer edges. Therefore, the model would detect the anomaly. However, in the alternative scenario, where the anomaly is purely due to the node degrees and the connections are between nodes that have a record of interaction, the alliGATOR model would not be able to detect the anomaly.

Another direction for further exploration is to test the ability of the alliGATOR model to reconstruct the topology of **larger graphs and detect multiple anomalous instances** simultaneously based on the reconstruction. This should be possible, as we build the node-guided topology reconstructor on the EDGE model. Chen et al. (2023) highlight the scalability of the EDGE model to graphs with thousands of nodes, due to the introduction of active nodes. We hypothesise that with an appropriately larger link predictor in the node-guided topology reconstructor, the alliGATOR model should be able to detect anomalous instances by calculating the anomaly score for all instances in the reconstructed graph. This would not only eliminate the extra step of calculating the personalised PageRank for each instance in the input graph, but would also allow for the extension

of the model to detect suspicious group behaviour, such as money laundering. This exciting prospect would require devising a measure that matches groups of low-probability edges with known money laundering patterns.

Immediate next steps include extending the alliGATOR model to incorporate **edge directionality**, which would increase its complexity, but also broaden its range of potential applications. This can be achieved in several ways. The simplest approach assumes the reconstructed graph can be a multigraph; allowing multiple directed edges between the same pair of nodes. In this case, the topology reconstructor could predict both directions between active nodes and sample them independently. Alternatively, active nodes can be split into senders and recipients, and edges could be predicted and sampled in a bipartite graph setting.

CHAPTER

FIVE

CONCLUSION

The aim of this thesis was to investigate the use of generative diffusion models for anomaly detection in graph-structured data. During our investigation, we identified a potentially significant research gap. Namely, that there was no unsupervised graph anomaly detection model for node and edge level anomalies which uses a generative method that models local properties. This prompted further exploration on whether node anomalies and edge anomalies are detectable via a comparison of original and reconstructed graph topologies.

Noting this gap, we sought to create a novel method rooted in the idea that node connections are intrinsic to their attributes. Therefore, we hypothesise that by reconstructing a graph's topology using the node attributes, and comparing the original edges with the reconstructed ones, we can detect anomalies on node and edge level. To evidence this notion we created the Graph Anomaly Detector through diffusion based Topology Reconstruction model – the alliGATOR model.

For the reconstruction of the topology, we were interested in modelling the local properties of the graph. We developed a discrete diffusion based topology reconstructor, guided by node attributes. The process of reconstruction begins with an empty graph containing only nodes with their attributes and degrees, and proceeds by incrementally adding edges. At each diffusion step, the node-guided topology reconstructor employs a graph neural network-based link predictor to generate probabilities for potential edges. A Bernoulli distribution, parameterised by each edge's probability, determines whether the edge is added to the graph.

To evaluate the capabilities of the alliGATOR model in detecting graph anomalies, we conducted two experiments: one focused on node-level anomaly detection, and the other on edge-level anomaly

detection. For this evaluation, we generated a synthetic dataset using a custom method we developed. Our synthetic data generation approach aims to model realistic graph structures, incorporating different node categories whose interactions follow power-law, uniform, or normal distributions. Anomalies were injected by either connecting node types that do not typically interact (for edge anomalies), or by altering a node's type such that its interaction pattern no longer aligns with its original category (for node anomalies). To assess the performance of the alliGATOR model, we analysed the precision-recall curve and reported the average precision. To provide context for the results, we compared the alliGATOR model against the Isolation Forest baseline. Both models were trained and evaluated under an inductive setting.

The results of the experiments demonstrate that the alliGATOR model effectively detects both node and edge anomalies. While our model outperforms Isolation Forest on the node anomaly detection task, both models exhibit near-perfect performance on the edge anomaly task. This indicates that the edge anomalies we designed may be too simplistic and do not require an extensive model like the alliGATOR.

Our work demonstrates that diffusion-based generative models can be effectively applied to graph anomaly detection, contributing a new unsupervised method to the field. In regard to future work, researchers may choose to focus on extending the model to incorporate edge directionality, exploring more complex anomaly types, evaluating its scalability on real-world datasets, and investigating its potential for detecting group-based anomalies such as coordinated fraud.

This model acts as an elementary and foundational step for researchers, financial institutions, banks, and governments in developing personalised autonomous anomaly detection systems. Additionally, the node-guided topology reconstructor may be extended to forecast future connections between nodes; the applicability of which would be wide and varied, not limited to finance. One example would be the facilitation of content provision in the entertainment industry, where forecasting may prove useful for user recommendation systems.

BIBLIOGRAPHY

Djihad Arrar, Nadjet Kamel, and Abdelaziz Lakhfif. A comprehensive survey of link prediction methods. *The journal of supercomputing*, 80(3):3902–3942, 2024.

Jacob Austin, Daniel D Johnson, Jonathan Ho, Daniel Tarlow, and Rianne Van Den Berg. Structured denoising diffusion models in discrete state-spaces. *Advances in Neural Information Processing Systems*, 34:17981–17993, 2021.

Albert-László Barabási and Réka Albert. Emergence of scaling in random networks. *science*, 286 (5439):509–512, 1999.

Vic Barnett and Toby Lewis. *Outliers in statistical data*, volume 3. Wiley New York, 1994.

Christopher M Bishop and Hugh Bishop. *Deep learning: Foundations and concepts*. Springer Nature, 2023.

Shaked Brody, Uri Alon, and Eran Yahav. How attentive are graph attention networks? In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

Xiaohui Chen, Jiaxing He, Xu Han, and Li-Ping Liu. Efficient and degree-guided graph generation via discrete diffusion modeling. In *Proceedings of the 40th International Conference on Machine Learning*, pages 4585–4610, 2023.

Kyunghyun Cho, Bart van Merriënboer, Çaglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014.

Fan Chung. A brief survey of pagerank algorithms. *IEEE Transactions on Network Science and Engineering*, 1(01):38–42, 2014.

Cifas and Royal United Services Institute. Cifas and rusi highlight growing threat to uk security from identity fraud, December 2024. URL `https://www.cifas.org.uk/newsroom/cifas-rusi-growing-id-fraud-threat`.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.

Prafulla Dhariwal and Alexander Nichol. Diffusion models beat gans on image synthesis. *Advances in neural information processing systems*, 34:8780–8794, 2021.

EBA and ECB. Eba/ecb report on payment fraud 2024. Technical report, European Banking Authority and European Central Bank, August 2024. URL `https://www.eba.europa.eu/sites/default/files/2024-08/465e3044-4773-4e9d-8ca8-b1cd031295fc/EBA_ECB%202024%20Report%20on%20Payment%20Fraud.pdf`.

Stefan Elfwing, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural networks*, 107:3–11, 2018.

Youssef Elmougy and Ling Liu. Demystifying fraudulent transactions and illicit nodes in the bitcoin network for financial forensics. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, page 3979–3990, 2023.

Paul Erdős and Alfréd Rényi. On the strength of connectedness of a random graph. *Acta Mathematica Hungarica*, 12(1):261–267, 1961.

Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning*, pages 1263–1272, 2017.

Markus Goldstein and Seiichi Uchida. A comparative evaluation of unsupervised anomaly detection algorithms for multivariate data. *PloS one*, 11(4):e0152173, 2016.

Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks. *Communications of the ACM*, 63(11):139–144, 2020.

Aditya Grover and Jure Leskovec. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 855–864, 2016a.

Aditya Grover and Jure Leskovec. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 855–864, 2016b.

Kilian Konstantin Haefeli, Karolis Martinkus, Nathanaël Perraudin, and Roger Wattenhofer. Diffusion models for graphs benefit from discrete state spaces. In *NeurIPS 2022 Workshop: New Frontiers in Graph Learning*, 2022.

Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30, 2017.

Douglas M Hawkins. *Identification of outliers*, volume 11. Springer, 1980.

Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. In *NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications*, 2021.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

Binxuan Huang and Kathleen M Carley. Residual or gate? towards deeper graph neural networks for inductive graph representation learning. *arXiv preprint arXiv:1904.08035*, 2019.

Han Huang, Leilei Sun, Bowen Du, Yanjie Fu, and Weifeng Lv. Graphgdp: Generative diffusion processes for permutation invariant graph generation. In *2022 IEEE International Conference on Data Mining (ICDM)*, pages 201–210, 2022a.

Xuanwen Huang, Yang Yang, Yang Wang, Chunping Wang, Zhisheng Zhang, Jiarong Xu, Lei Chen, and Michalis Vazirgiannis. Dgraph: A large-scale financial dataset for graph anomaly detection. *Advances in Neural Information Processing Systems*, 35:22765–22777, 2022b.

Minqi Jiang, Chaochuan Hou, Ao Zheng, Xiyang Hu, Songqiao Han, Hailiang Huang, Xiangnan He, Philip S Yu, and Yue Zhao. Weakly supervised anomaly detection: A survey. *arXiv preprint arXiv:2302.04549*, 2023.

Jaehyeong Jo, Seul Lee, and Sung Ju Hwang. Score-based generative modeling of graphs via the system of stochastic differential equations. In *International conference on machine learning*, pages 10362–10383, 2022.

Jaehyeong Jo, Dongki Kim, and Sung Ju Hwang. Graph generation with diffusion mixture. In *International Conference on Machine Learning*, pages 22371–22405, 2024.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In Yoshua Bengio and Yann LeCun, editors, *3rd International Conference on Learning Representations*, 2015.

Diederik P. Kingma and Max Welling. Auto-Encoding Variational Bayes. In *2nd International Conference on Learning Representations*, 2014.

Thomas N. Kipf and Max Welling. Semi-Supervised Classification with Graph Convolutional Networks. In *Proceedings of the 5th International Conference on Learning Representations*, 2017.

Hang Li, Wei Jin, Geri Skenderi, Harry Shomer, Wenzhuo Tang, Wenqi Fan, and Jiliang Tang. Subgraph based diffusion model for link prediction. In *Proceedings of the Third Learning on Graphs Conference (LoG)*, 2024.

Xujia Li, Yuan Li, Xueying Mo, Hebing Xiao, Yanyan Shen, and Lei Chen. Diga: guided diffusion model for graph recovery in anti-money laundering. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 4404–4413, 2023.

Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *2008 eighth ieee international conference on data mining*, pages 413–422. IEEE, 2008.

Dawid Malarz, Artur Kasymov, Maciej Zieba, Jacek Tabor, and Przemysław Spurek. Classifier-free guidance with adaptive scaling. *arXiv preprint arXiv:2502.10574*, 2025.

Annamalai Narayanan, Mahinthan Chandramohan, Rajasekar Venkatesan, Lihui Chen, Yang Liu, and Shantanu Jaiswal. graph2vec: Learning distributed representations of graphs. In *13th International Workshop on Mining and Learning with Graphs*, 2017.

Mark Newman. *Networks*. Oxford university press, 2018.

Chenhao Niu, Yang Song, Jiaming Song, Shengjia Zhao, Aditya Grover, and Stefano Ermon. Permutation invariant graph generation via score-based generative modeling. In *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, pages 4474–4484, 2020.

Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The pagerank citation ranking: Bringing order to the web. Technical report, Stanford infolab, 1999.

Ioana Pintilie, Andrei Manolache, and Florin Brad. Time series anomaly detection using diffusion-based models. In *2023 IEEE International Conference on Data Mining Workshops (ICDMW)*, pages 570–578. IEEE, 2023.

Leonardo F.R. Ribeiro, Pedro H.P. Saverese, and Daniel R. Figueiredo. struc2vec: Learning node representations from structural identity. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2017.

Bernhard Schölkopf and Alexander J Smola. *Learning with kernels: support vector machines, regularization, optimization, and beyond*. MIT press, 2002.

Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *Proceedings of the 32nd International Conference on Machine Learning*, pages 2256–2265, 2015.

Sumsub. Identity fraud report 2024, 2024. URL `https://sumsub.com/fraud-report-2024/`.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. Graph attention networks. *arXiv preprint arXiv:1710.10903*, 2017.

Clement Vignac, Igor Krawczuk, Antoine Siraudin, Bohan Wang, Volkan Cevher, and Pascal Frossard. Digress: Discrete denoising diffusion for graph generation. In *The Eleventh International Conference on Learning Representations*, 2023.

Xu-Wen Wang, Yize Chen, and Yang-Yu Liu. Link prediction through deep generative model. *iScience*, 23(10):101626, 2020.

Julia Wolleb, Florentin Bieder, Robin Sandkühler, and Philippe C Cattin. Diffusion models for medical anomaly detection. In *International Conference on Medical image computing and computer-assisted intervention*, pages 35–45. Springer, 2022.

Shiwen Wu, Fei Sun, Wentao Zhang, Xu Xie, and Bin Cui. Graph neural networks in recommender systems: a survey. *ACM Computing Surveys*, 55(5):1–37, 2022.

Xuan Xia, Xizhou Pan, Nan Li, Xing He, Lin Ma, Xiaoguang Zhang, and Ning Ding. Gan-based anomaly detection: A review. *Neurocomputing*, 493:497–535, 2022.

Appendices

APPENDIX

A

EDGE FORMULA DERIVATIONS

The derivation of the loss function from Chapter 2, Section 2.4, Eq. 2.19 is presented on the next page.

$$
\begin{aligned}
&\mathcal{L}(A^0;\theta) = \log p_\theta(A^0) \\
&= \log \int p(A^{0:T}, s^{1:T}) dA^{1:T} && \text{Marginalize by int.} \\
&= \log \int p_\theta(A^{0:T}, s^{1:T}) \frac{q(A^{1:T}, s^{1:T}|A^0)}{q(A^{1:T}, s^{1:T}|A^0)} dA^{1:T} && \text{Multiply and divide } q. \\
&= \log \int q(A^{1:T}, s^{1:T}|A^0) \frac{p_\theta(A^{0:T}, s^{1:T})}{q(A^{1:T}, s^{1:T}|A^0)} dA^{1:T} && \text{Rearrange terms} \\
&= \log \mathbb{E}_{q(A^{1:T}, s^{1:T}|A^0)} \Big[\frac{p_\theta(A^{0:T}, s^{1:T})}{q(A^{1:T}, s^{1:T}|A^0)} \Big] && \text{Definition of expectation} \\
&\geq \mathbb{E}_q \Big[\log \frac{p_\theta(A^{0:T}, s^{1:T})}{q(A^{1:T}, s^{1:T}|A^0)} \Big] && \text{Jensen's inequality} \\
&= \mathbb{E}_q \Big[\log \frac{p(A^T) \prod_{t=1}^T p_\theta(A^{t-1}, s^t|A^t)}{\prod_{t=1}^T q(A^t, s^t|A^{t-1})} \Big] && \text{Section 2.4.2 and Eq. 2.11} \\
&= \mathbb{E}_q \Big[\log p(A^T) + \sum_{t=1}^T \log \frac{p_\theta(A^{t-1}|A^t, s^t) p_\theta(s^t|A^t)}{q(A^t|A^{t-1}), q(s^t|A^t, A^{t-1})} \Big] && \text{Eq.2.17 and 2.18} \\
&= \mathbb{E}_q \Big[\log p(A^T) + \log \frac{p_\theta(A^0|A^1, s^1) p_\theta(s^1|A^1)}{q(A^1|A^0), q(s^1|A^1, A^0)} \\
&\quad + \sum_{t=2}^T \log \frac{p_\theta(A^{t-1}|A^t, s^t) p_\theta(s^t|A^t)}{\frac{q(A^{t-1}|A^t, s^t, A^0) q(A^t|A^0) q(s^t|A^t, A^0)}{q(A^{t-1}|A^0)}} \Big] && \text{Posterior of forward process with actives} \\
&= \mathbb{E}_q \Big[\log p(A^T) + \log \frac{p_\theta(A^0|A^1, s^1) p_\theta(s^1|A^1)}{q(A^1|A^0), q(s^1|A^1, A^0)} \\
&\quad + \sum_{t=2}^T \log \frac{p_\theta(A^{t-1}|A^t, s^t) p_\theta(s^t|A^t) q(A^{t-1}|A^0)}{q(A^{t-1}|A^t, s^t, A^0) q(A^t|A^0) q(s^t|A^t, A^0)} \Big] && \text{Rearrange terms} \\
&= \mathbb{E}_q \Big[\log \frac{p(A^T) \cancel{q(A^T|A^0)}}{q(A^T|A^0)} + \log \frac{p_\theta(A^0|A^1, s^1) p_\theta(s^1|A^1)}{\cancel{q(A^1|A^0)}, q(s^1|A^1, A^0)} \\
&\quad + \sum_{t=2}^T \log \frac{p_\theta(A^{t-1}|A^t, s^t) p_\theta(s^t|A^t) \cancel{q(A^{t-1}|A^0)}}{q(A^{t-1}|A^t, s^t, A^0) q(s^t|A^t, A^0) \cancel{q(A^t|A^0)}} \Big] && \text{Multiply and divide } q. \\
&= \mathbb{E}_q \Big[\log \frac{p(A^T)}{q(A^T|A^0)} + \underbrace{\log p_\theta(A^0|A^1, s^1)}_{\text{reconstruction term } \mathcal{L}_{rec}} \\
&\quad + \sum_{t=2}^T \underbrace{\log \frac{p_\theta(A^{t-1}|A^t, s^t)}{q(A^{t-1}|A^t, s^t, A^0)}}_{\text{edge prediction term } \mathcal{L}_{edge}(t)} + \sum_{t=1}^T \underbrace{\log \frac{p_\theta(s^t|A^t)}{q(s^t|A^t, A^0)}}_{\text{node selection term } \mathcal{L}_{node}(t)} \Big] && \text{Cancelling and rearrangement}
\end{aligned}
\tag{A.1}
$$